TEKsystems Global Services | WORKFORCE DEVELOPMENT

QUALITY ASSURANCE INTRODUCTION



DAY 2: AUTOMATION

- Test-driven development: red, green, refactor
- Why and what to automate
- Your first automated UI test with Cypress
- UI testing: locators, visual regression, accessibility, and cross-browser
- API testing with Postman, contracts, and mocks

Yesterday you designed tests. Today you make them run themselves.

© 2026 by Innovation In Software Corporation

FROM TEST CASES TO CODE

Yesterday's test cases were written for people: steps, data, and expected results. Today the same ideas become code that runs in seconds, every time, without anyone clicking.

The thinking doesn't change. A good automated test still has a clear purpose, exact data, and a specific expected result. It's still traceable to an acceptance criterion. Only the executor changes.



© 2026 by Innovation In Software Corporation

WHAT IS A UNIT TEST?

A unit test checks one small piece of code, usually a single function, in isolation from the rest of the system. It calls the function with known input and checks the output.

- Runs in milliseconds, so you can run hundreds on every change
- When it fails, it points to exactly one function
- Lives next to the code, in the same repository
- The base of a healthy test suite



© 2026 by Innovation In Software Corporation

ANATOMY OF A UNIT TEST: ARRANGE, ACT, ASSERT

Most unit tests follow the same three steps.

```
def test_percent_code_takes_percentage_off():
    subtotal = 7998                                  # Arrange: set up the input
    discount = calculate_discount(subtotal, "SAVE15", TODAY)   # Act: call the code
    assert discount == 1200                          # Assert: check the result
```

- **Arrange:** the data and state the test needs
- **Act:** one call to the code under test
- **Assert:** one clear expected result
- The test's **name** says what behavior it proves

© 2026 by Innovation In Software Corporation

UNIT TEST FRAMEWORKS

Every major language has a unit test framework. The ideas are identical; only the syntax changes.

Language	Framework	A test looks like	Run with
Python	**pytest**	`def test_...():` `assert ...`	`python -m pytest`
JavaScript	**Jest**	`test('...', () =>` `expect(...).toBe(...))`	`npm test`
Java	**JUnit**	`@Test void ...()` `{ assertEquals(...); }`	`mvn test` or `gradle test`

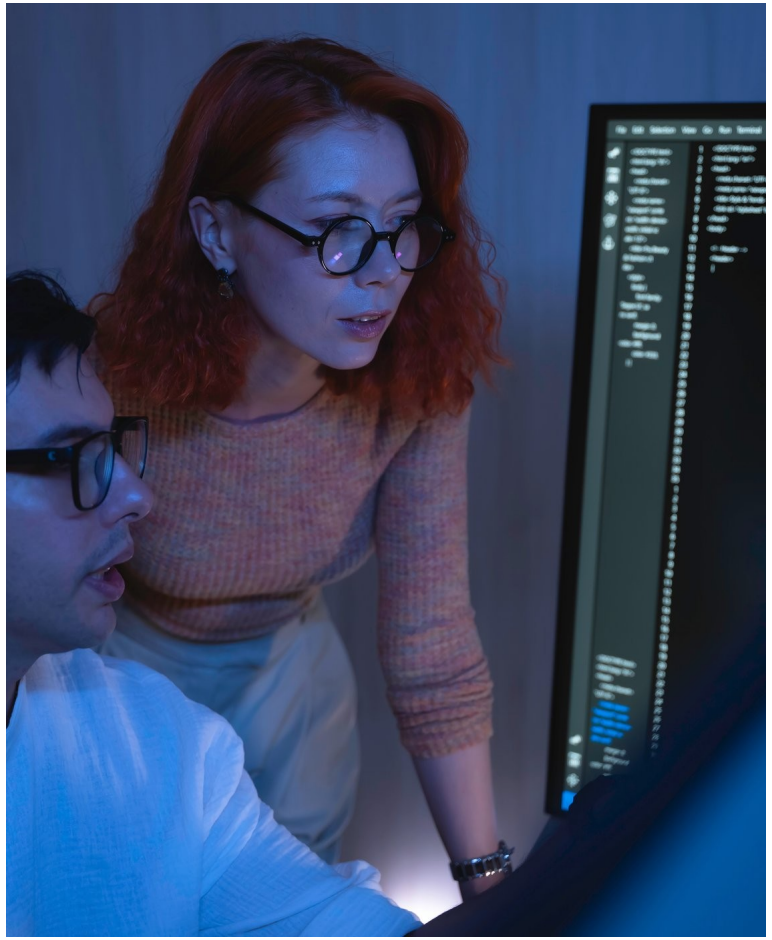
In the lab you'll pick Python or JavaScript. Everything you learn transfers directly to Java and JUnit.

© 2026 by Innovation In Software Corporation

WHAT IS TEST-DRIVEN DEVELOPMENT?

Test-driven development (TDD) flips the usual order. Instead of writing code and then testing it, you write a **failing test first**, then write just enough code to make it pass.

The test becomes a small, precise specification. The code exists only to satisfy it. Repeat in small steps, and the code and its tests grow up together.



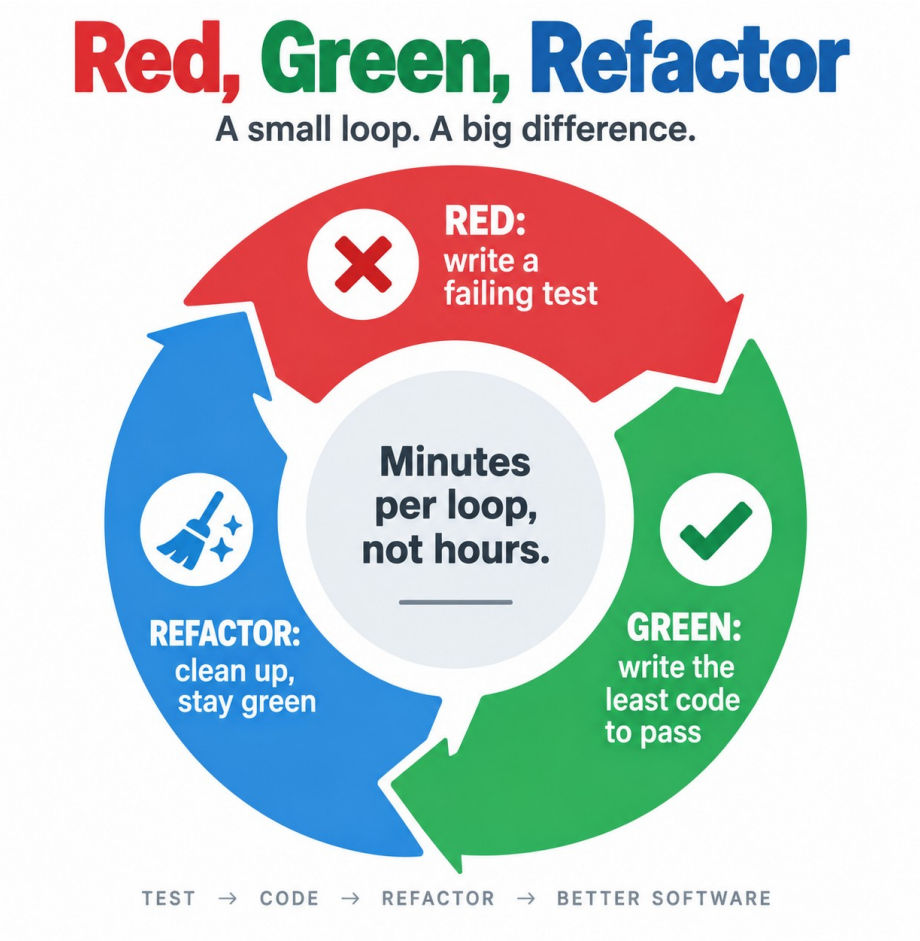
© 2026 by Innovation In Software Corporation

THE RED-GREEN-REFACTOR CYCLE

TDD is a loop of three short steps, each a few minutes long.

- **Red:** write one small test that fails
- **Green:** write the least code that makes it pass
- **Refactor:** clean up the code and tests, staying green

•Repeat with the next small piece of behavior



© 2026 by Innovation In Software Corporation

RED: WRITE A FAILING TEST

Start with one small piece of behavior from the spec, and write a test for it before the code exists.

- Pick the **simplest** next behavior, not the hardest
- Run it and **watch it fail**, for the right reason
- "Module not found" is a valid first red: the code doesn't exist yet

```
def test_no_code_gives_no_discount():
    assert calculate_discount(7998, "", TODAY)
== 0
```



© 2026 by Innovation In Software Corporation

GREEN: THE SIMPLEST CODE THAT PASSES

Now write the **least** code that makes the test pass. Not the full feature, not the clever version: just enough.

```
def calculate_discount(subtotal, code, today=None):
    return 0
```

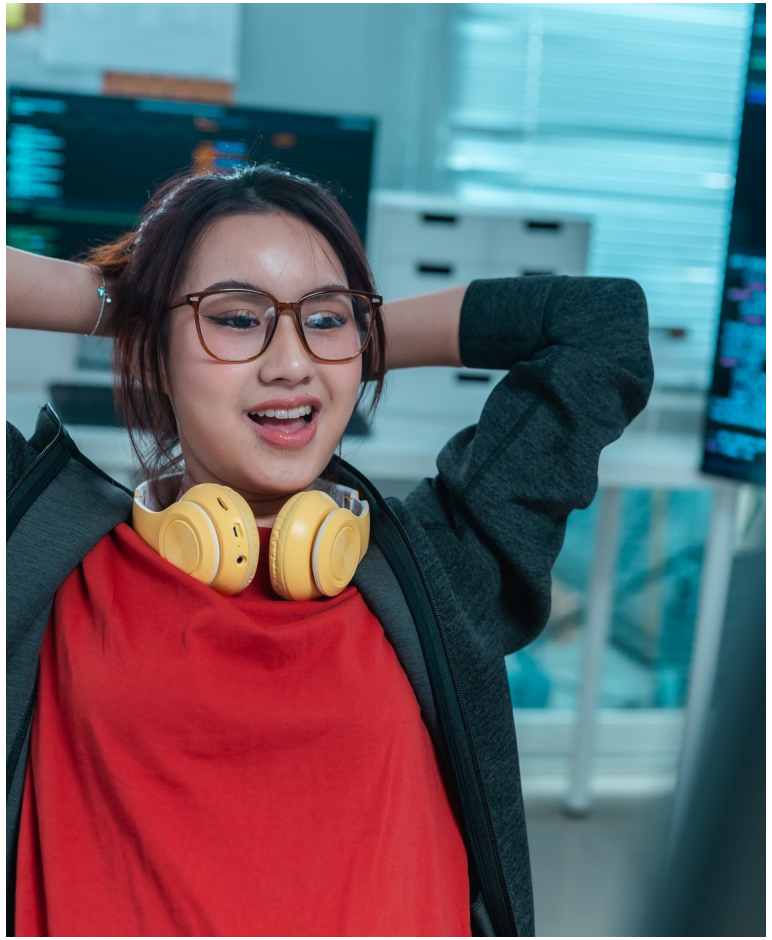
Yes, `return 0` is a fake. That's fine. The next test will force real logic. Writing more than the tests demand means writing code nothing checks.

© 2026 by Innovation In Software Corporation

REFACTOR: CLEAN UP WHILE GREEN

With all tests passing, improve the code without changing what it does: remove duplication, name magic numbers, simplify.

- Run the tests after every change
- If anything goes red, undo the last change
- Refactor tests too: clear names, no copy-paste
- The passing tests are what make refactoring safe



© 2026 by Innovation In Software Corporation

WHY WATCH THE TEST FAIL FIRST?

A test you've never seen fail might not be testing anything. It could be checking the wrong function, asserting something that's always true, or never running at all.

Seeing it fail proves two things: the test **can** catch a problem, and the behavior really wasn't there yet. Then green means something.

© 2026 by Innovation In Software Corporation

SMALL STEPS

TDD works because each step is tiny. You're never more than a few minutes from working code.

- One behavior per test, one test at a time
- When a step feels too big, write a smaller test
- If you're stuck in red for long, go back to the last green
- Commit often; each green is a safe point



© 2026 by Innovation In Software Corporation

FAKE IT, THEN TRIANGULATE

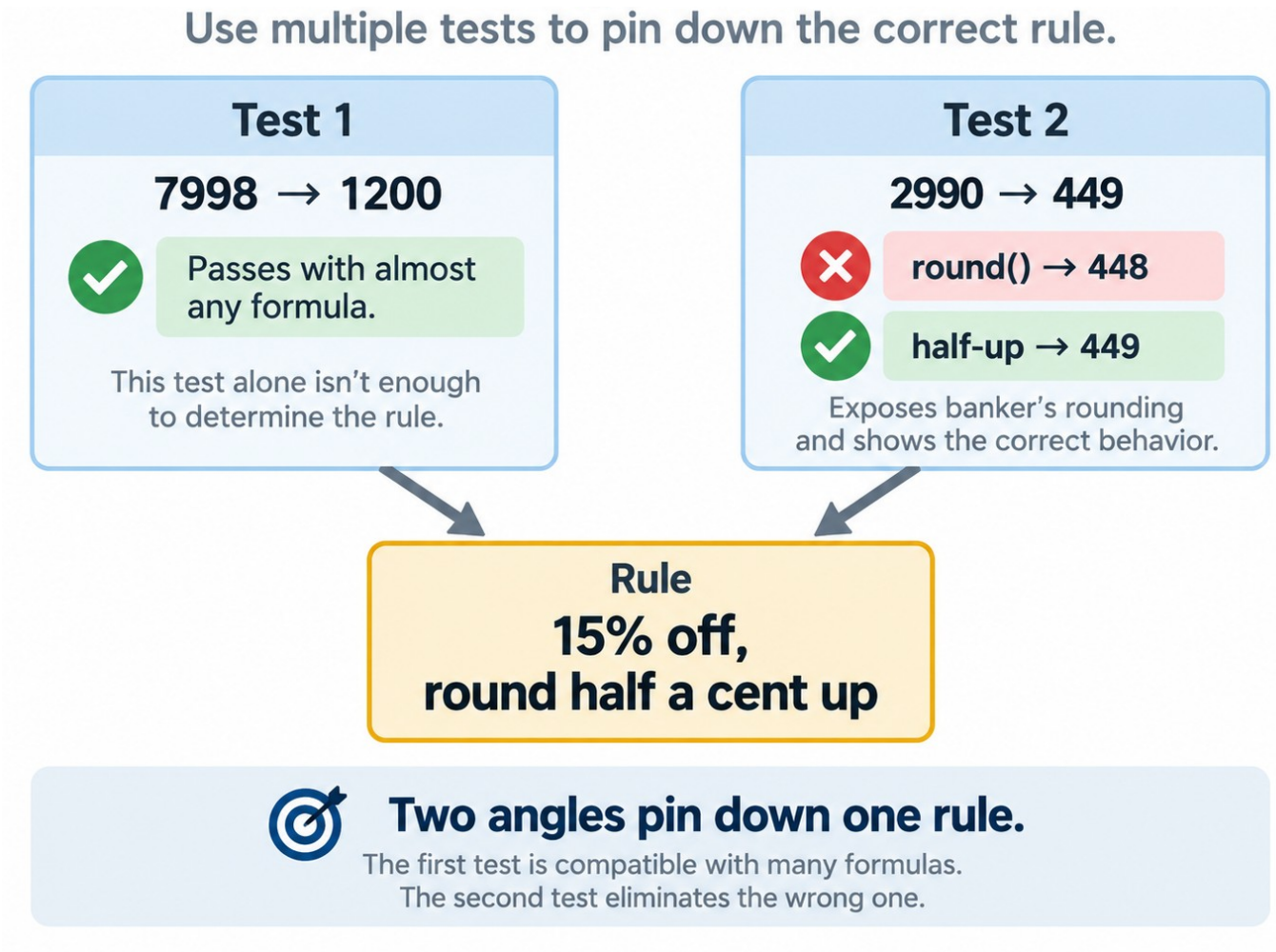
One passing example can hide a bug. **Triangulate**: add a second example that checks the same rule from a different angle, so a fake or a lucky formula can't pass both.

Test	Input	Expected	Why it matters
First example	15% of $79.98	$12.00	Works with almost any rounding
Triangulation	15% of $29.90 (448.5¢)	$4.49 (rounds half up)	Exposes the wrong rounding rule

Money, rounding, and limits are where triangulation pays off most. Example assumes Python

© 2026 by Innovation In Software Corporation

FAKE IT, THEN TRIANGULATE (PYTHON)



© 2026 by Innovation In Software Corporation

PARAMETERIZED TESTS

When the same rule applies to many inputs, write one test and feed it a list of values. Each value runs and reports as its own test.

```
@pytest.mark.parametrize("code", ["save15", "Save15", " SAVE15 "])
def test_code_ignores_case_and_spaces(code):
    assert calculate_discount(7998, code, TODAY) == 1200
```

```
test.each(['save15', 'Save15', ' SAVE15 '])('code "%s" ignores case and spaces', code => {
  expect(calculateDiscount(7998, code, TODAY)).toBe(1200);
});
```

It's the automated version of Day 1's test data tables.

© 2026 by Innovation In Software Corporation

CONTROL TIME AND OTHER RANDOMNESS

A test that depends on the real clock passes today and fails next year. The same goes for random numbers, the network, or anything else outside the test's control.

- Pass "today" in as a parameter instead of reading the clock inside the function
- Tests then choose the date: the day before expiry, the day of, the day after
- Same input, same result, on every machine, every day



© 2026 by Innovation In Software Corporation

TESTING ERRORS

Rejecting bad input is behavior too, and it deserves tests. Check both that an error happens and that it's the **right** error, with the right message.

```
def test_unknown_code_is_rejected():
    with pytest.raises(InvalidCodeError, match="That code isn't valid."):
        calculate_discount(7998, "BOGUS", TODAY)
```

```
expect(() => calculateDiscount(7998, 'BOGUS', TODAY)).toThrow("That code isn't
valid.");
```

© 2026 by Innovation In Software Corporation

BENEFITS OF TDD

Teams that practice TDD consistently report the same advantages.

- Every behavior has a test, because the test came first
- Fewer bugs reach testers and users
- Refactoring is safe, so code stays clean
- Code written to be testable tends to be simpler and less tangled
- The test names read like a specification of the feature



© 2026 by Innovation In Software Corporation

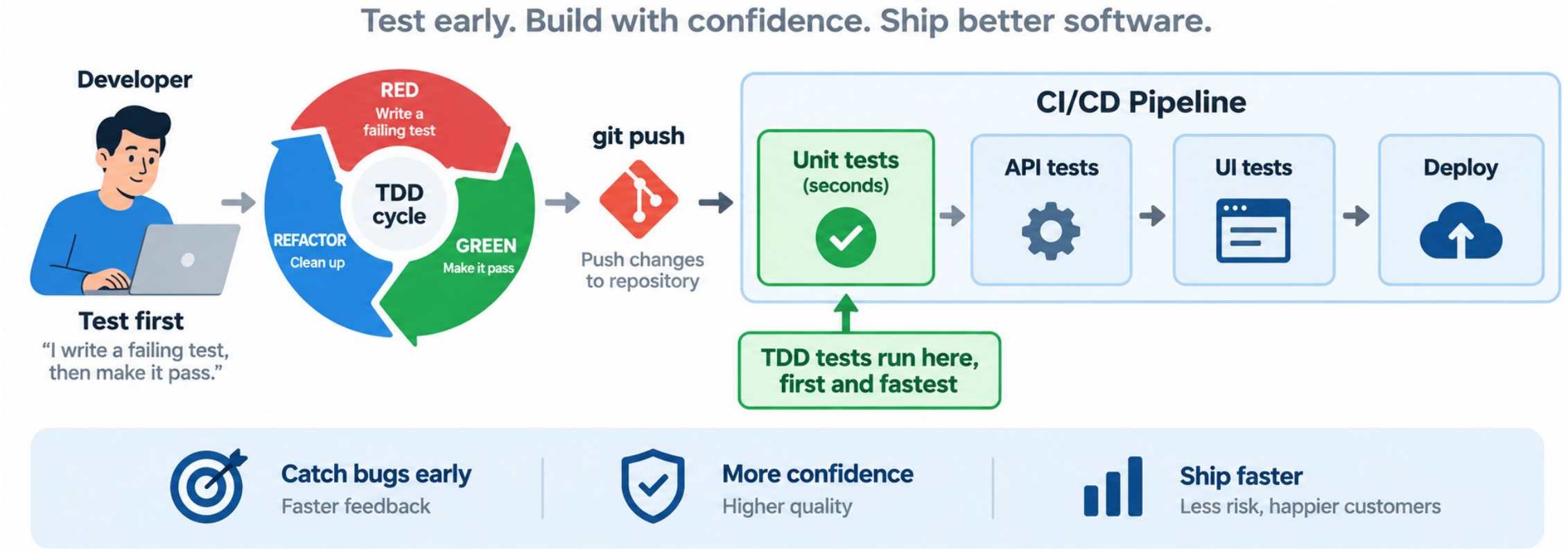
TDD IN AGILE AND DEVOPS

TDD fits naturally with short sprints and continuous delivery. Small, tested steps are exactly what a pipeline needs.

- **Agile:** each story's acceptance criteria become tests early in the sprint
- **DevOps:** the unit tests run on every change in seconds and block bad merges
- Fast feedback means a mistake is caught minutes after it's made
- On Day 3 your TDD tests become the first stage of a CI pipeline

© 2026 by Innovation In Software Corporation

TDD IN AGILE AND DEVOPS



© 2026 by Innovation In Software Corporation

TESTS AS LIVING DOCUMENTATION

Good test names describe behavior in plain language. Run the suite in verbose mode and it reads like the spec, and unlike a document, it can't fall out of date without someone noticing.

- `TC-3.1 percent code takes percentage off`
- `TC-3.3 code is valid on its expiry date`
- `TC-3.5 discount never exceeds subtotal`

Naming tests after their test case IDs keeps the thread from Day 1's acceptance criteria all the way to code.



© 2026 by Innovation In Software Corporation

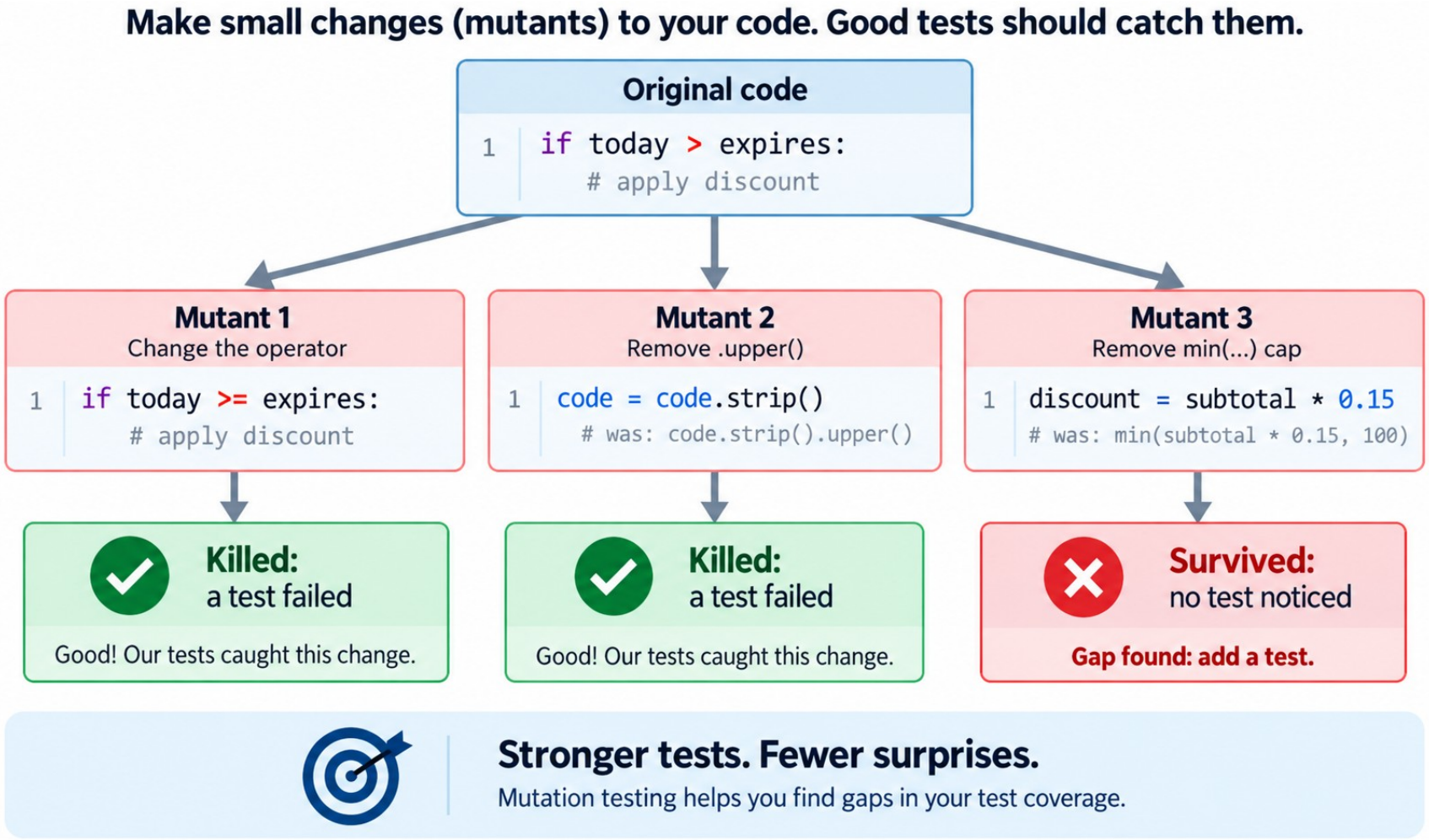
MUTATION TESTING: TESTING YOUR TESTS

How do you know your tests are good enough? Break the code on purpose and see whether a test notices. Each deliberate break is a **mutant**.

- Change > to >=, remove a check, flip a condition
- A test goes red: the mutant is **killed**, so your tests caught it
- Everything stays green: the mutant **survived**, so you've found a gap
- Tools can do this automatically; today you'll do it by hand

© 2026 by Innovation In Software Corporation

MUTATION TESTING: TESTING YOUR TESTS



© 2026 by Innovation In Software Corporation

TDD MYTHS AND LIMITS

Myth	Reality
"TDD means no other testing"	Unit tests can't see integration, UI, or usability problems
"TDD is slower"	Slower to start, usually faster overall: less debugging and rework
"100% of code must be test-driven"	Use it where logic matters most: rules, calculations, edge cases
"Only developers do TDD"	Testers shape the tests: which cases, which boundaries, which errors

© 2026 by Innovation In Software Corporation

LAB 2.1 TDD RED GREEN REFACTOR

In this lab you build a discount calculator for BookNook from scratch, test first, using Python or JavaScript.

- Follow three guided red-green-refactor cycles
- Drive the rest of the spec yourself: fixed codes, case and spaces, invalid and expired codes, and the subtotal cap
- Use parameterized tests and control the date in your tests
- Break your own code on purpose to check that your tests catch it

© 2026 by Innovation In Software Corporation

POP QUIZ:

What is the correct order of the TDD cycle?

A. Write code, write a test, refactor

B. Refactor, write a test, write code

C. Write a failing test, write the least code to pass, refactor

D. Write all the tests, then write all the code





POP QUIZ:

What is the correct order of the TDD cycle?

A. Write code, write a test, refactor

B. Refactor, write a test, write code

C. Write a failing test, write the least code to pass, refactor

D. Write all the tests, then write all the code





POP QUIZ:

Why does the discount function take "today" as a parameter instead of reading the system clock?

A. It runs faster

B. So tests control the date and give the same result every day

C. Python can't read the system clock

D. So the function can only be used in tests





POP QUIZ:

Why does the discount function take "today" as a parameter instead of reading the system clock?

A. It runs faster

B. So tests control the date and give the same result every day

C. Python can't read the system clock

D. So the function can only be used in tests





POP QUIZ:

You change > to >= in your code, and every test still passes. What does that tell you?

A. The change was harmless, so keep it

B. The test framework is broken

C. Mutation testing doesn't work on comparisons

D. Your tests have a gap at that boundary





POP QUIZ:

You change > to >= in your code, and every test still passes. What does that tell you?

A. The change was harmless, so keep it

B. The test framework is broken

C. Mutation testing doesn't work on comparisons

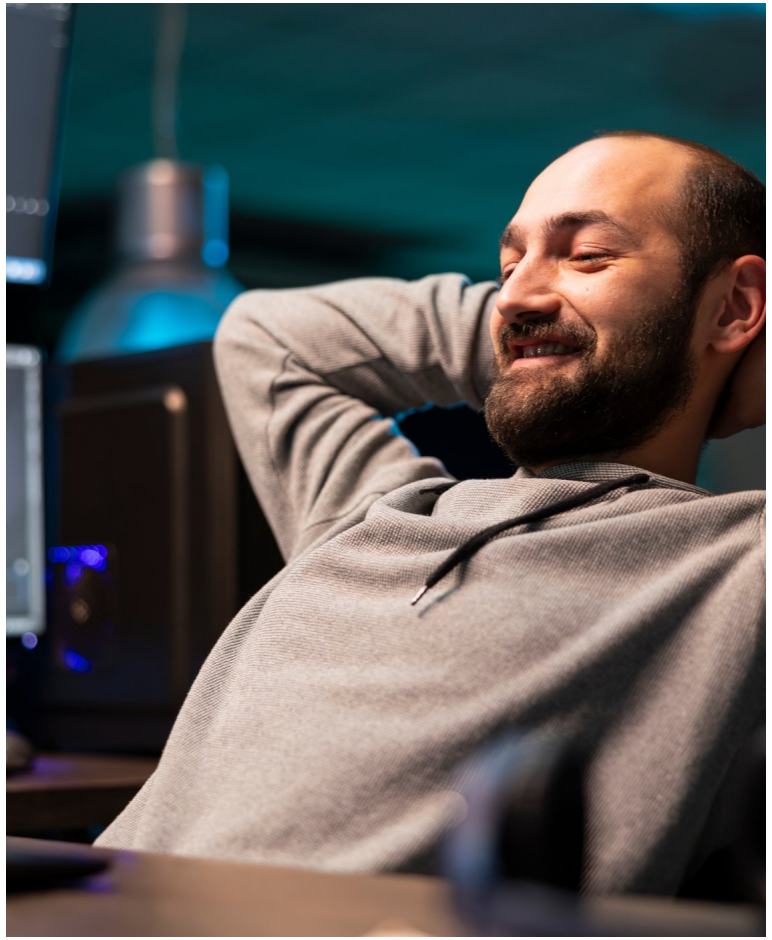
D. Your tests have a gap at that boundary



WHY AUTOMATE TESTING?

Every release needs the same core checks: can people log in, search, add to the cart, check out? Running them by hand after every change is slow, tiring, and error-prone. People skip steps when they're tired. Machines don't.

Automation turns those repeated checks into code that runs in minutes, on every change, day or night. That frees testers for the work machines can't do: exploring, questioning, and judging.



© 2026 by Innovation In Software Corporation

THE REGRESSION BURDEN

Every new feature adds tests that must keep passing. Run by hand, the regression suite grows every sprint until it takes days, and teams start cutting corners.

- Sprint 1: 20 manual checks, an hour
- Sprint 10: 300 manual checks, several days
- Without automation, either releases slow down or regression gets skipped



© 2026 by Innovation In Software Corporation

WHAT TO AUTOMATE, WHAT TO KEEP MANUAL

Automate when a test is...	Keep manual when a test is...
Run often (every build, every deploy)	Run once or rarely
High-risk if it breaks (login, money)	Exploratory, or about "how it feels"
Stable: the feature isn't changing weekly	On a UI that's still being redesigned
Tedious or data-heavy (many inputs)	Hard to check by machine (visual appeal, wording)
Needed as a gate in CI/CD	Cheaper to run by hand than to build and maintain

© 2026 by Innovation In Software Corporation

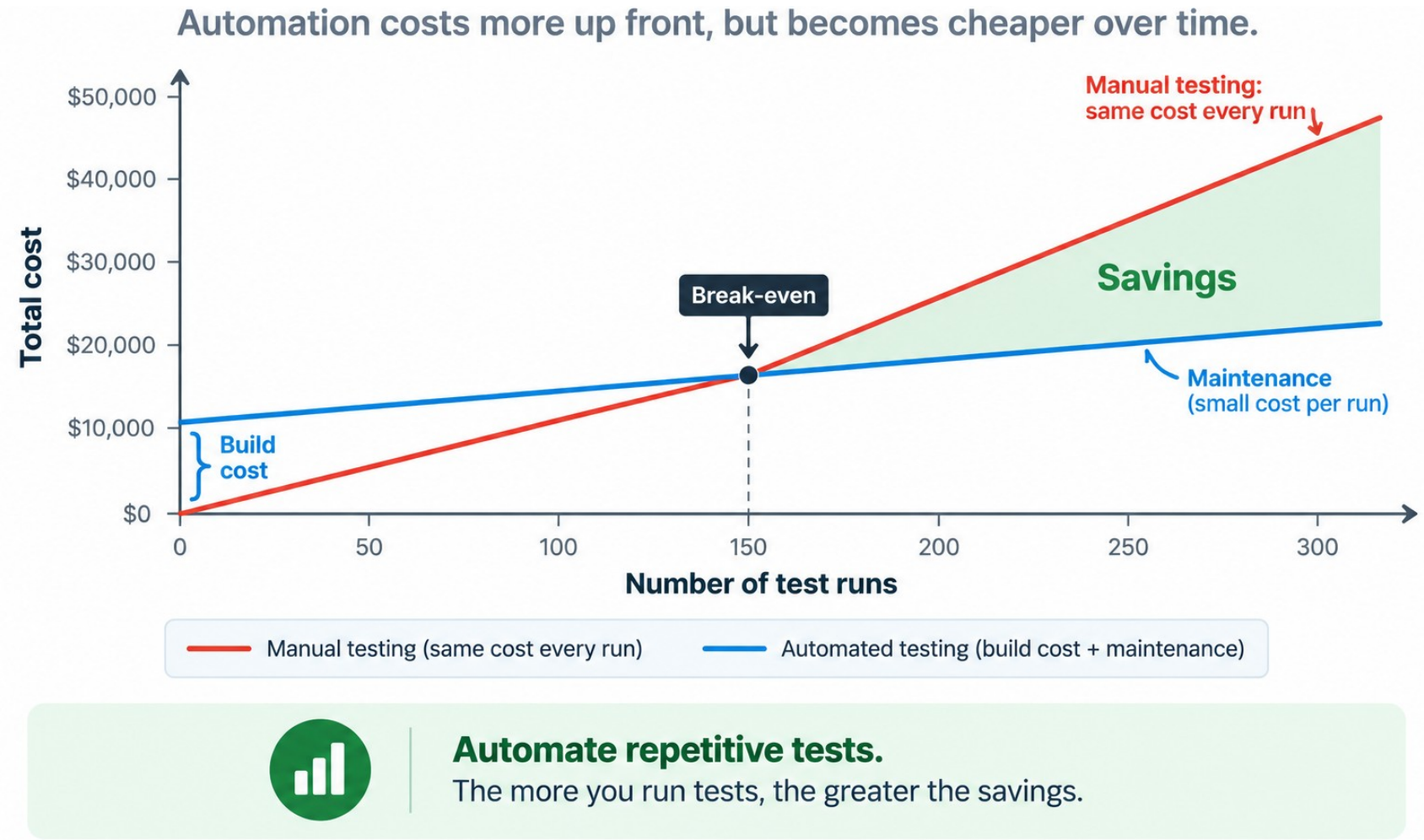
AUTOMATION ISN'T FREE

An automated test costs time to **build** and keeps costing time to **maintain** as the app changes. It pays off through **repetition**: every run after the first is nearly free.

- A test that runs once is cheaper by hand
- A test that runs on every commit pays for itself within weeks
- Brittle tests that break on every UI change can cost more than they save

© 2026 by Innovation In Software Corporation

AUTOMATION ISN'T FREE



© 2026 by Innovation In Software Corporation

TYPES OF AUTOMATED TESTS

Type	Tests	Speed	BookNook example	Where in this course
Unit	One function	Milliseconds	`calculate_discount`	Lab 2.1
API	Endpoints, no browser	Fast	`POST /api/auth/login` returns 401	Lab 2.4
UI (end-to-end)	The app through a real browser	Slow	Log in and see your email in the header	Labs 2.2 and 2.3
Performance	Speed under load	Varies	Search under 3 s with 200 users (NF1)	Lab 2.4, a first look

© 2026 by Innovation In Software Corporation

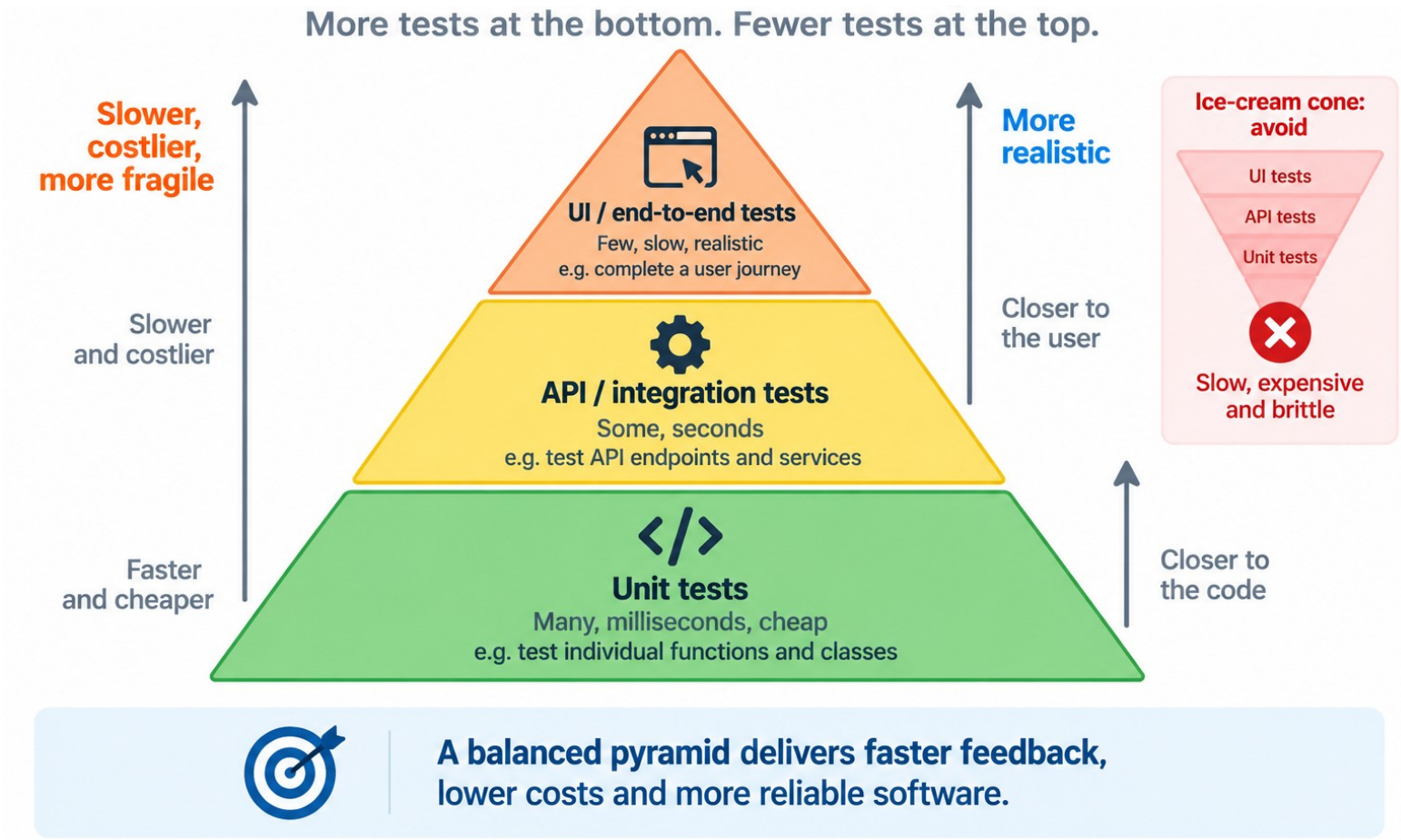
THE TEST AUTOMATION PYRAMID

Healthy test suites have **many** fast unit tests, **some** API tests, and **few** slow UI tests. The higher you go, the slower, costlier, and more fragile the tests get, but the more of the real system they cover.

- Push each check to the **lowest** level that can prove it
- Use UI tests for what only the UI can show
- An upside-down pyramid, with mostly UI tests, is slow and flaky

© 2026 by Innovation In Software Corporation

THE TEST AUTOMATION PYRAMID



© 2026 by Innovation In Software Corporation

PERFORMANCE TESTS IN AN AUTOMATED SUITE

Performance tests are automated too, but they're a different kind of tool and run on a different schedule.

- **Functional suites** check "is it right?" on every change
- **Performance suites** check "is it fast enough under load?" before releases or nightly
- Simple response-time checks can run with functional tests as an early warning
- Real load testing needs dedicated tools, such as k6 or JMeter, and a production-like environment

© 2026 by Innovation In Software Corporation

CHOOSING A TEST AUTOMATION TOOL

There's no single best tool. The right one fits your team, your app, and your pipeline.

- What language does the team already write?
- Which browsers and devices must we support?
- Does it run headless in our CI system?
- How easy is it to debug a failure?
- Can the team maintain it for years, not weeks?



© 2026 by Innovation In Software Corporation

SELENIUM, CYPRESS, AND PLAYWRIGHT

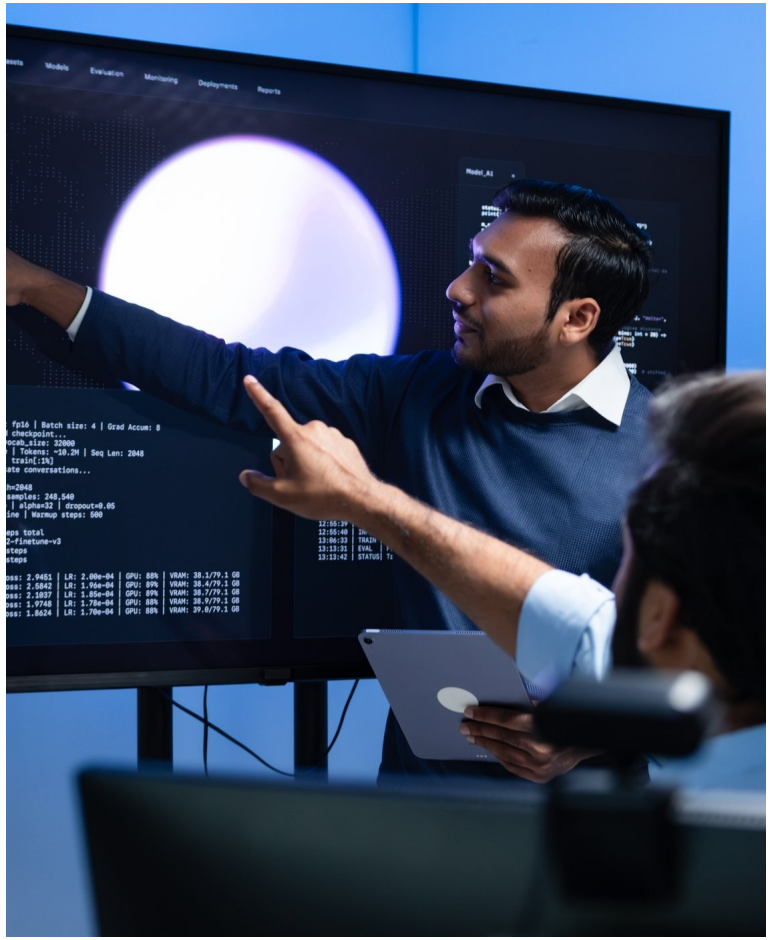
	Selenium	Cypress	Playwright
Languages	Java, Python, C#, JS, more	JavaScript/TypeScript	JS/TS, Python, Java, C#
Browsers	All major	Chrome, Edge, Firefox, Electron	Chrome, Edge, Firefox, WebKit (Safari engine)
Waiting	Explicit waits you write	Automatic retries	Automatic waits
Debugging	Logs, screenshots	Interactive runner with time travel	Trace viewer
Best fit	Large multi-language orgs, existing suites	Web apps, fast feedback, JS teams	Cross-browser coverage, multi-language teams

© 2026 by Innovation In Software Corporation

SELENIUM WEBDRIVER

Selenium is the long-standing industry standard. Your test code sends commands through **WebDriver**, a standard protocol, to a driver that controls a real browser.

- Works with almost every language and browser
- Huge community and many existing enterprise suites
- You manage waits yourself, which is a common source of flaky tests
- Same concepts as Cypress: find an element, act on it, check the result



© 2026 by Innovation In Software Corporation

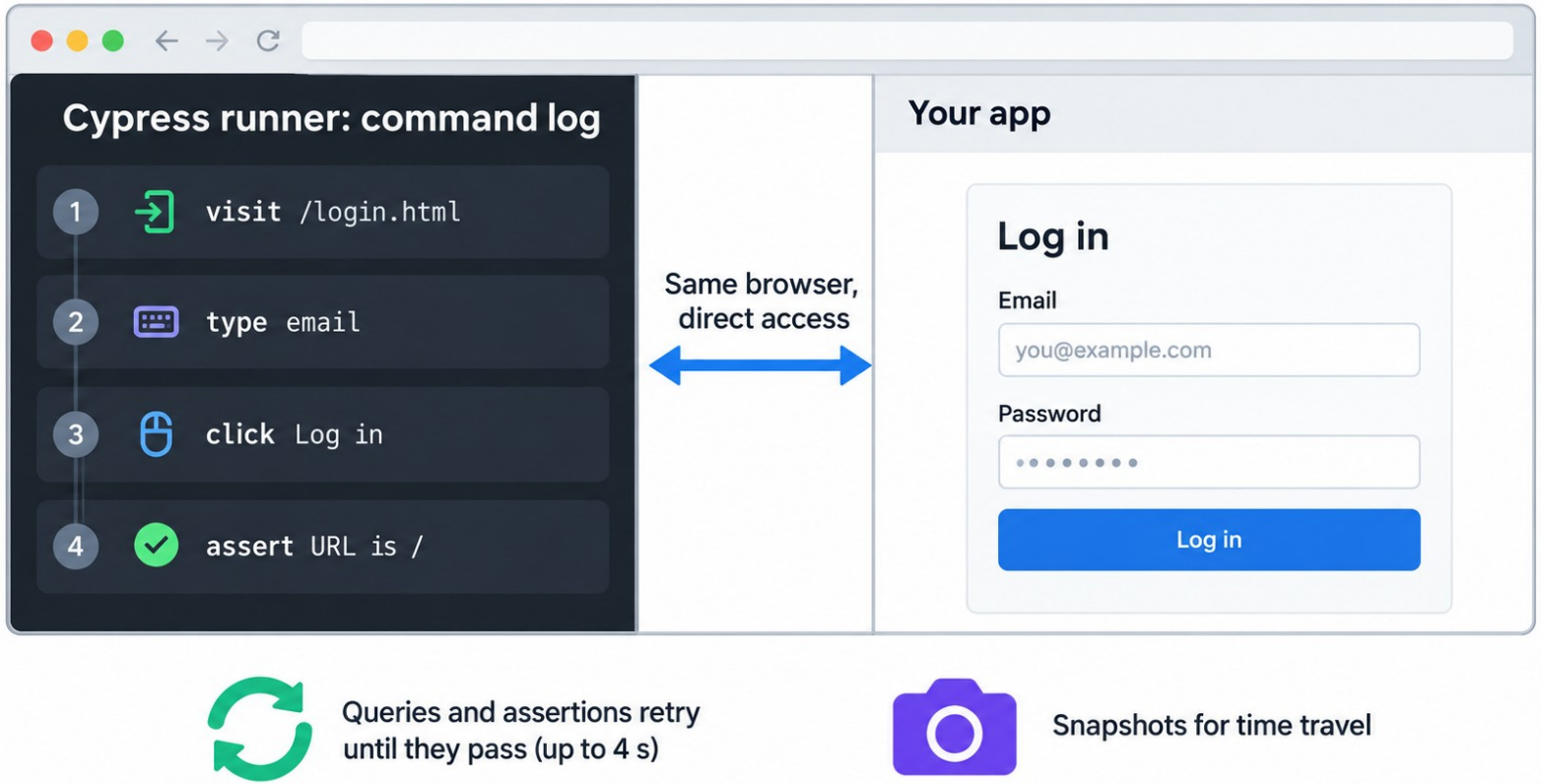
HOW CYPRESS WORKS

Cypress runs **inside** the browser, right next to your app. That gives it direct access to the page, the network, and the timing of everything.

- Commands queue up and run in order
- Every query and assertion **retries** automatically until it passes or times out
- The runner records every step so you can time-travel through a test
- Tests are JavaScript, and they run headless in CI

© 2026 by Innovation In Software Corporation

HOW CYPRESS WORKS



© 2026 by Innovation In Software Corporation

ANATOMY OF A CYPRESS TEST

```
describe('US1 – Log in', () => {
  it('TC-1.1 valid login takes user to catalog', () => {
    cy.visit('/login.html');

    cy.get('#email').type('qa.user@booknook.test');
    cy.get('#password').type('Passw0rd!');
    cy.get('#login-btn').click();

    cy.location('pathname').should('eq', '/');
  });
});
```

- `describe` groups tests by story; `it` is one test case
- The test name keeps the **TC ID** from Day 1
- Arrange (visit), act (type, click), assert (`should`)

© 2026 by Innovation In Software Corporation

CORE CYPRESS COMMANDS

Command	Does
`cy.visit(path)`	Opens a page
`cy.get(selector)`	Finds element(s) by CSS selector
`cy.contains(selector, text)`	Finds an element containing text
`.type(text)` / `.click()` / `.clear()`	Acts like a user
`.should(assertion, value)`	Checks a result, **retrying** until it passes or times out
`cy.location('pathname')`	Reads the current URL path
`cy.request(method, url)`	Calls an API directly, without the browser UI

© 2026 by Innovation In Software Corporation

FROM MANUAL TEST CASE TO AUTOMATED TEST

A well-written manual test case translates almost line for line.

TC-1.1 (Day 1)	Cypress
Precondition: test user exists	Reset data before the test
Step 1: go to the login page	`cy.visit('/login.html')`
Steps 2–3: enter credentials, click Log in	`.type(...)`, `.click()`
Expected: catalog shown	`cy.location('pathname').should('eq', '/')`
Expected: header shows the email	`cy.get('#user-email').should('have.text', ...)`

Vague test cases make vague automated tests. Day 1's precision pays off here.

© 2026 by Innovation In Software Corporation

ASSERTIONS THAT RETRY

Web pages don't update instantly. After a click, the app calls the server and redraws the page a moment later. An assertion that checks once, too early, fails for no real reason.

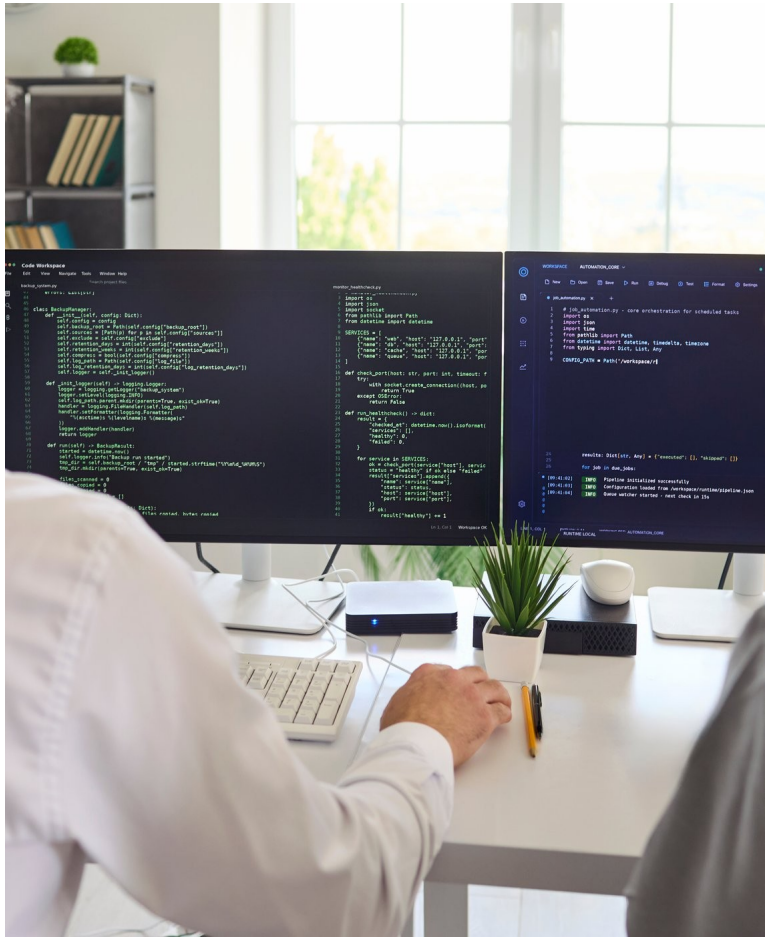
Cypress's `.should()` keeps rechecking for up to four seconds and passes as soon as the condition is true. You almost never need a fixed `wait`. Fixed waits are too slow when the app is fast and still too short when it's slow.

© 2026 by Innovation In Software Corporation

INTERACTIVE AND HEADLESS RUNS

Cypress runs two ways, for two jobs.

- **Interactive** (`npx cypress open`): watch the test run in a browser, click any step, see the page at that moment. Best for writing and debugging.
- **Headless** (`npx cypress run`): no window, a summary table in the terminal, a screenshot for every failure. Best for CI pipelines.



© 2026 by Innovation In Software Corporation

MAKE IT FAIL ON PURPOSE

Before trusting a new automated test, break it. Change the expected value, run it, and read the failure.

- Does it fail for the reason you expected?
- Does the message show **expected vs. actual** clearly?
- Is there a screenshot or snapshot of the moment it failed?

A test you've never seen fail is a test you can't trust.



© 2026 by Innovation In Software Corporation

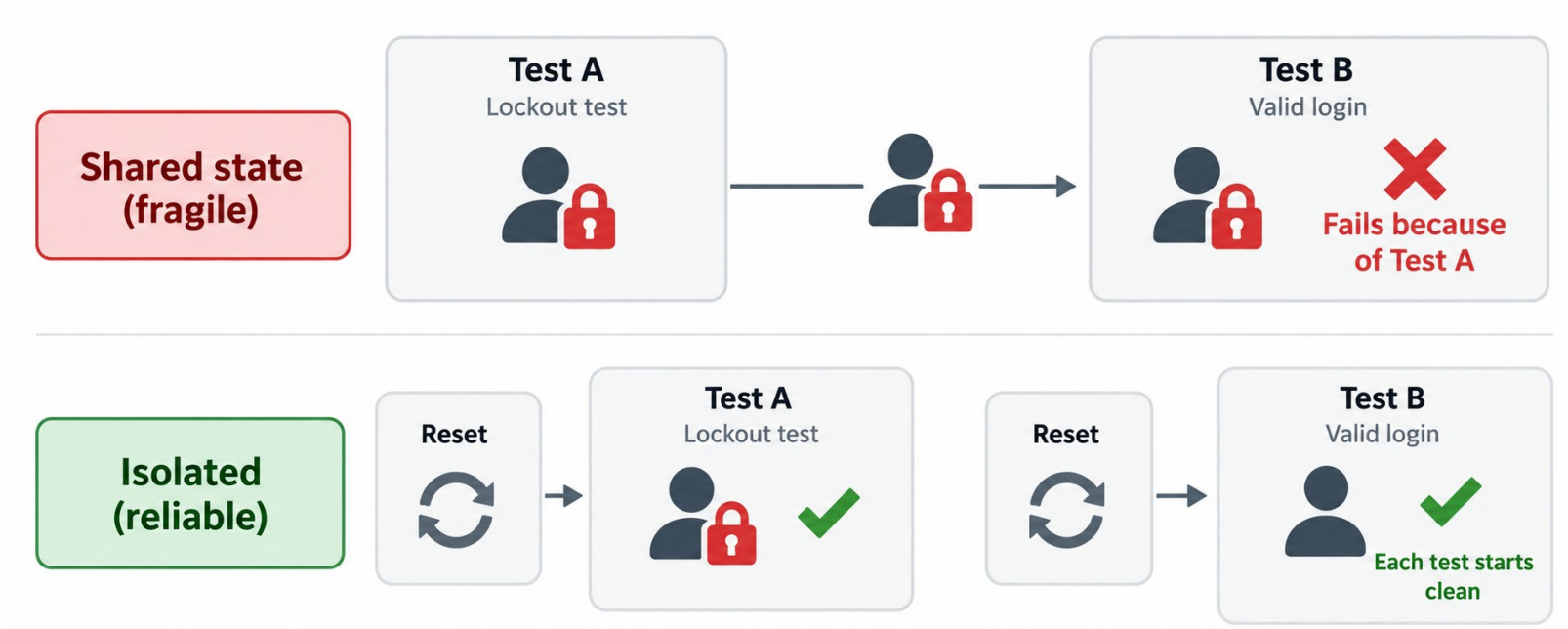
TEST ISOLATION: START FROM A KNOWN STATE

Every test must set up what it needs and must not depend on any other test. If one test locks an account and the next test logs in with it, the second fails for reasons that have nothing to do with its code.

- Reset data before **each** test, not once per file
- Never rely on the order tests run in
- Each test should pass when run alone

© 2026 by Innovation In Software Corporation

TEST ISOLATION: START FROM A KNOWN STATE



© 2026 by Innovation In Software Corporation

KEEP TESTS MAINTAINABLE

Automated tests are code, and they need the same care as product code.

```
function login(email, password) {
  cy.get('#email').clear().type(email);
  cy.get('#password').clear().type(password);
  cy.get('#login-btn').click();
}
```

- Put repeated steps in **helper functions**: if the form changes, you fix one place
- Keep each test focused on one behavior
- Name tests so a failure explains itself

© 2026 by Innovation In Software Corporation

DATA-DRIVEN TESTS

When several inputs should behave the same way, put them in a list and generate one test per row, just like Day 1's test data tables.

```
const badLogins = [
  { name: 'wrong password', email: USER.email, password: 'WrongPass1' },
  { name: 'unknown email', email: 'nobody@booknook.test', password: USER.password }
];

badLogins.forEach(({ name, email, password }) => {
  it(`TC-1.2 ${name} shows a generic error`, () => { /* ... */ });
});
```

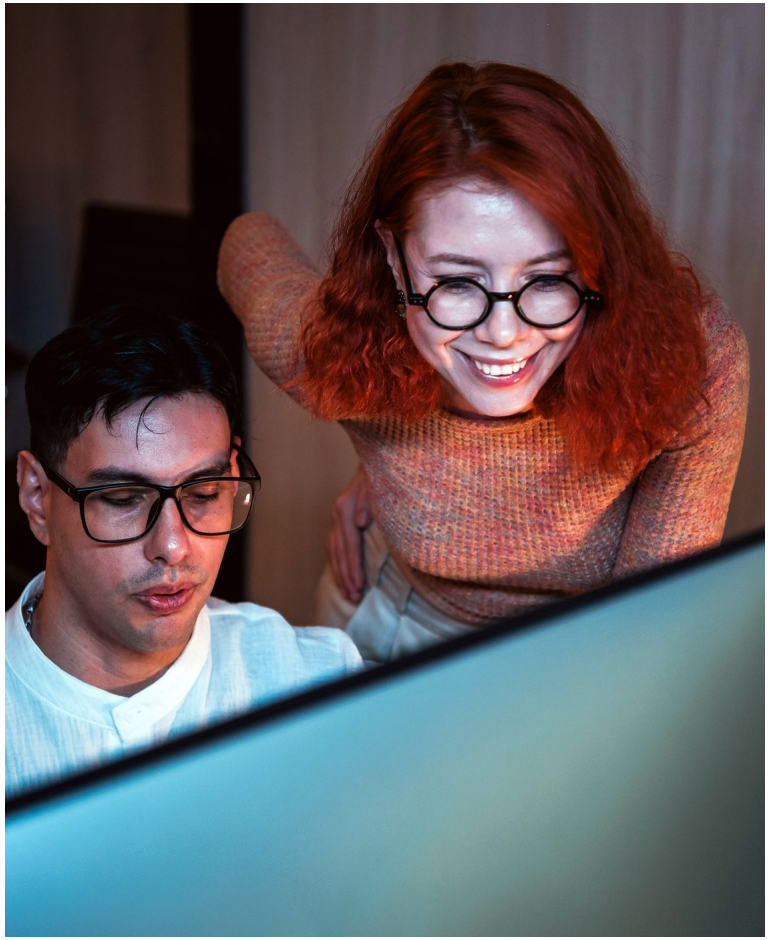
© 2026 by Innovation In Software Corporation

SECURITY TESTS IN DISGUISE

Some functional tests protect security, too. Recognizing them helps you see what's covered and what isn't.

Test	Protects against
The same message for a wrong email and a wrong password	**Account enumeration**: learning which emails have accounts
Lockout after 5 failed attempts	**Password guessing** (brute force)
The session cookie is **HttpOnly**	Scripts injected into a page stealing the login

Some security requirements, such as hashed passwords and HTTPS everywhere, need code reviews, configuration checks, and scanners instead.



© 2026 by Innovation In Software Corporation

DECIDING WHAT TO AUTOMATE

Score each test case before you automate it. High scores across the board make strong candidates.

- **Run often?** Every build beats once a quarter
- **Risk if broken?** Login and money beat the footer
- **Stable feature?** Automating a moving target wastes effort
- **Easy to automate?** Clear steps and exact expected results

Then pick the **cheapest level** that can prove it: unit first, then API, then UI.



© 2026 by Innovation In Software Corporation

LAB 2.2 YOUR FIRST AUTOMATED TEST

In this lab you automate BookNook's login test cases from Day 1 with Cypress, then decide what else deserves automation.

- Install Cypress and run your first test, TC-1.1, in the interactive runner
- Make it fail on purpose, then run headless the way CI will
- Add data reset, a login helper, and data-driven negative tests
- Automate the account lockout and check a security property of the session cookie
- Score other test cases to decide what to automate, and at which level

© 2026 by Innovation In Software Corporation

POP QUIZ:

Which test case is the strongest candidate for automation?

A. A one-time check of a marketing page before it's deleted

B. Judging whether a new color scheme looks good

C. An exploratory session on a brand-new feature

D. A login test that must pass after every deploy



POP QUIZ:

Which test case is the strongest candidate for automation?

A. A one-time check of a marketing page before it's deleted

B. Judging whether a new color scheme looks good

C. An exploratory session on a brand-new feature

D. A login test that must pass after every deploy



POP QUIZ:

According to the test automation pyramid, which kind of test should you have the most of?

A. UI end-to-end tests

B. Manual tests

C. Unit tests

D. Performance tests





POP QUIZ:

According to the test automation pyramid, which kind of test should you have the most of?

A. UI end-to-end tests

B. Manual tests

C. Unit tests

D. Performance tests





POP QUIZ:

Why does each Cypress test reset BookNook's data in beforeEach?

A. To make the tests run faster

B. So each test starts from a known state and doesn't depend on other tests

C. Cypress requires it before every visit

D. To clear the browser's cache



POP QUIZ:

Why does each Cypress test reset BookNook's data in beforeEach?

A. To make the tests run faster

B. So each test starts from a known state and doesn't depend on other tests

C. Cypress requires it before every visit

D. To clear the browser's cache



BEYOND "DOES IT WORK?"

Two weeks before launch, BookNook's product owner has three new worries. A CSS change broke a product page last release, and nobody noticed for a week. Legal says the site must meet accessibility standards. And half the traffic comes from phones, across four browsers.

Functional tests wouldn't catch any of these. The button still works; it just looks wrong, can't be reached by keyboard, or falls off a small screen.



© 2026 by Innovation In Software Corporation

WHAT UI TESTING CHECKS

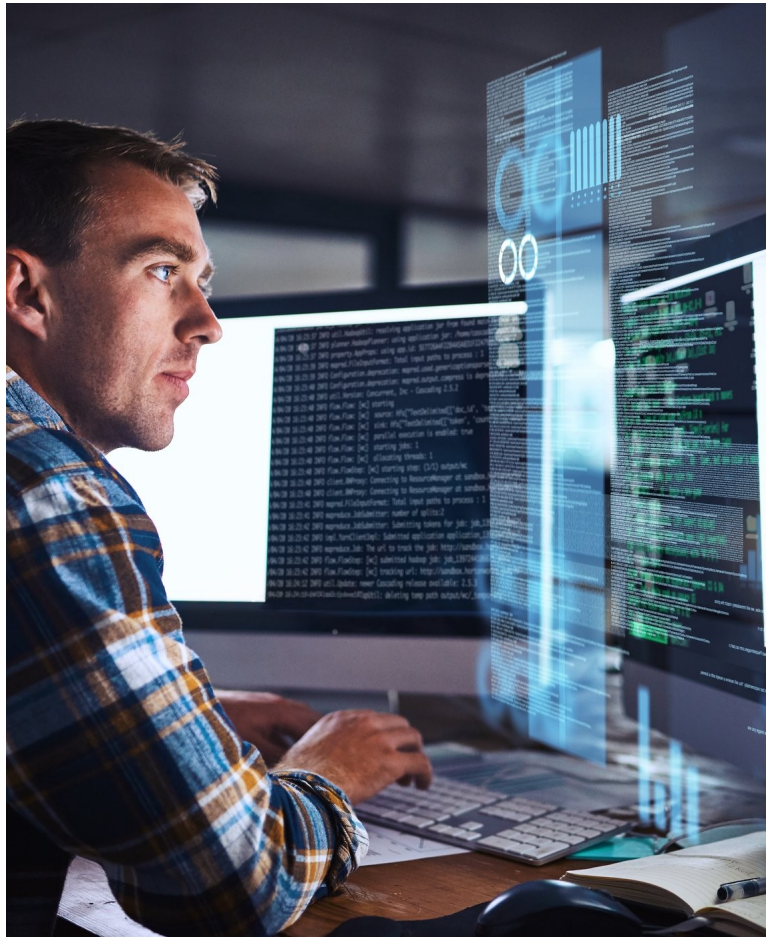
Kind	Question	Tool in the lab
Functional	Does the UI **do** the right thing?	Cypress
Visual	Does it **look** the way it did?	`cypress-visual-regression`
Accessibility	Can **everyone** use it, including keyboard and screen reader users?	`cypress-axe` (axe-core)
Cross-browser and device	Does it work in **every** supported browser and screen size?	Cypress `--browser`, `cy.viewport()`

© 2026 by Innovation In Software Corporation

FUNDAMENTALS OF UI TESTING

A UI test acts like a user and checks what a user would see. Beyond "the right page loaded," good UI tests check the details users depend on.

- **Content:** the right text, numbers, and messages
- **State:** buttons enabled or disabled at the right times
- **Navigation:** links and redirects go where they should
- **Feedback:** errors, empty states, and confirmations appear when they should
- **Layout:** nothing hidden, overlapping, or cut off



© 2026 by Innovation In Software Corporation

WHY UI TESTS BREAK

UI tests break most often because a **selector** stopped matching, not because the app broke. A designer moves a button into a new container, and fifty tests fail overnight while the feature works fine.

Every false failure costs time and trust. Choosing selectors that survive harmless changes is the single biggest factor in a maintainable UI suite.

© 2026 by Innovation In Software Corporation

CHOOSING LOCATORS

Prefer selectors tied to **meaning**, not **layout**.

Priority	Locator	Example	Breaks when...
Best	Dedicated test attribute	`[data-cy=search-input]`	Almost never; it exists only for tests
Good	Visible text	`cy.contains('button', 'Log in')`	The wording changes (often a real change)
OK	Stable `id`	`#email`	Someone renames it
Avoid	Styling classes	`.btn-primary`	The design changes
Avoid	Position or structure	`#results > div:nth-child(3) > button`	Anything is added, removed, or reordered

© 2026 by Innovation In Software Corporation

TEST ATTRIBUTES

A test attribute is a label developers add **only** for tests. It doesn't affect styling or behavior, so nobody changes it by accident.

```
<input data-cy="search-input" type="search" placeholder="Search title or author">
<button data-cy="add-to-cart">Add to Cart</button>
```

```
cy.contains('[data-cy=book-card]', 'Clean Code')
  .find('[data-cy=add-to-cart]')
  .click();
```

Asking developers for test attributes is a great example of QA and dev working together.

© 2026 by Innovation In Software Corporation

TESTING UI STATES

Many UI bugs hide in states other than "page loaded with data." Test each state a screen can be in.

State	Check
Empty	A helpful message, not a blank area ("No books found.")
One / many	Lists, counts, and paging ("Page 1 of 2")
Disabled	Buttons that shouldn't be used are disabled (no **Previous** on page 1)
Error	A clear message the user can act on
Loading	The test waits for the real content before checking

© 2026 by Innovation In Software Corporation

WHAT IS VISUAL REGRESSION TESTING?

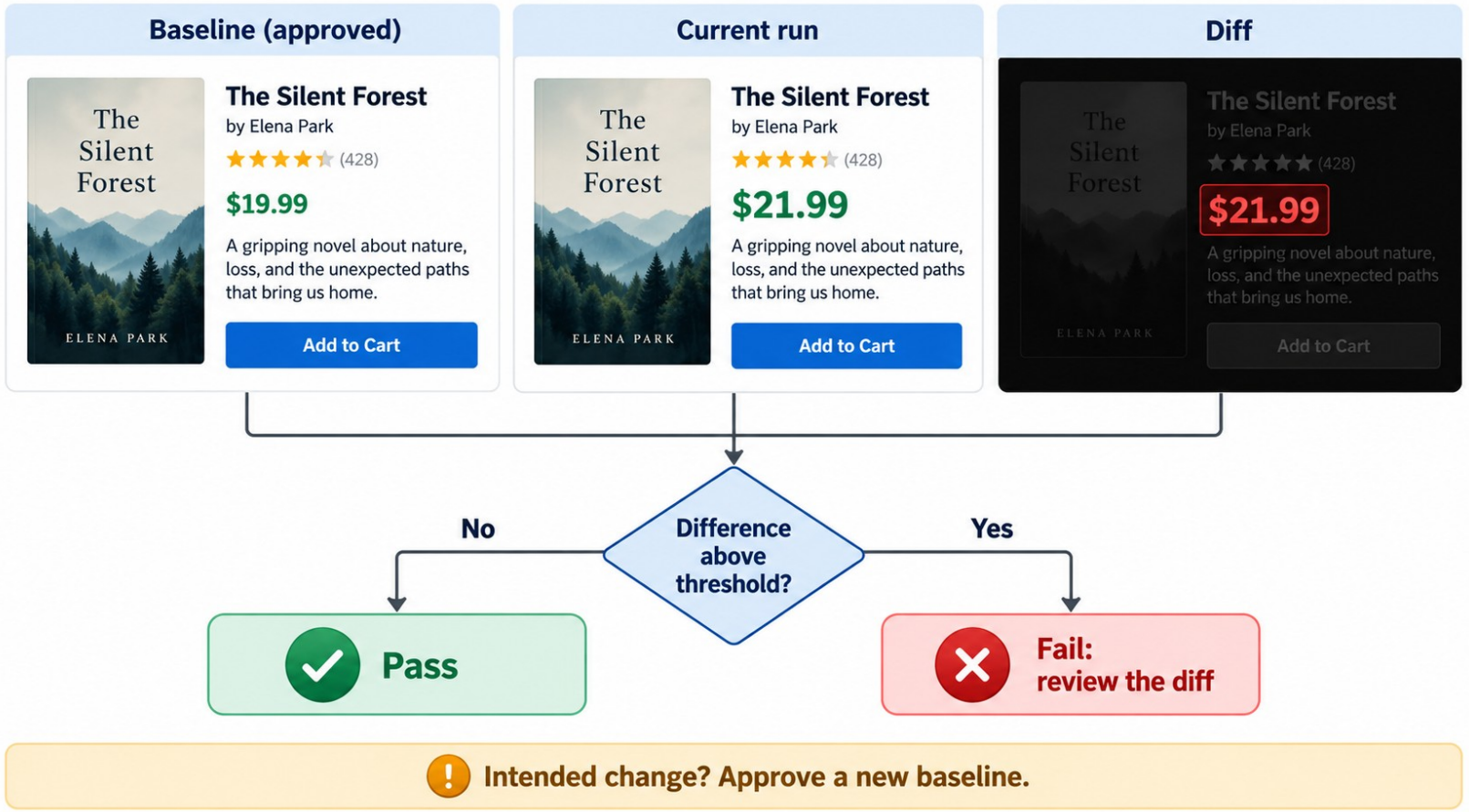
A visual test takes a screenshot and compares it, pixel by pixel, with an approved **baseline** image. Any difference above a threshold fails the test.

It catches what functional tests miss: a price in the wrong color, overlapping text, a missing image, a layout shift. These are the changes that make a site look broken even when every button still works.



© 2026 by Innovation In Software Corporation

HOW VISUAL COMPARISON WORKS



© 2026 by Innovation In Software Corporation

BASELINES AND APPROVING CHANGES

A baseline is the "approved look" of a page. Visual tests compare every run against it.

- Create baselines deliberately, review them, and commit them with the code
- When a test fails, open the **diff** image first
- **Unintended** change: it's a bug, so report it
- **Intended** change: don't "fix" the test; approve a new baseline

© 2026 by Innovation In Software Corporation

VISUAL TESTING PITFALLS

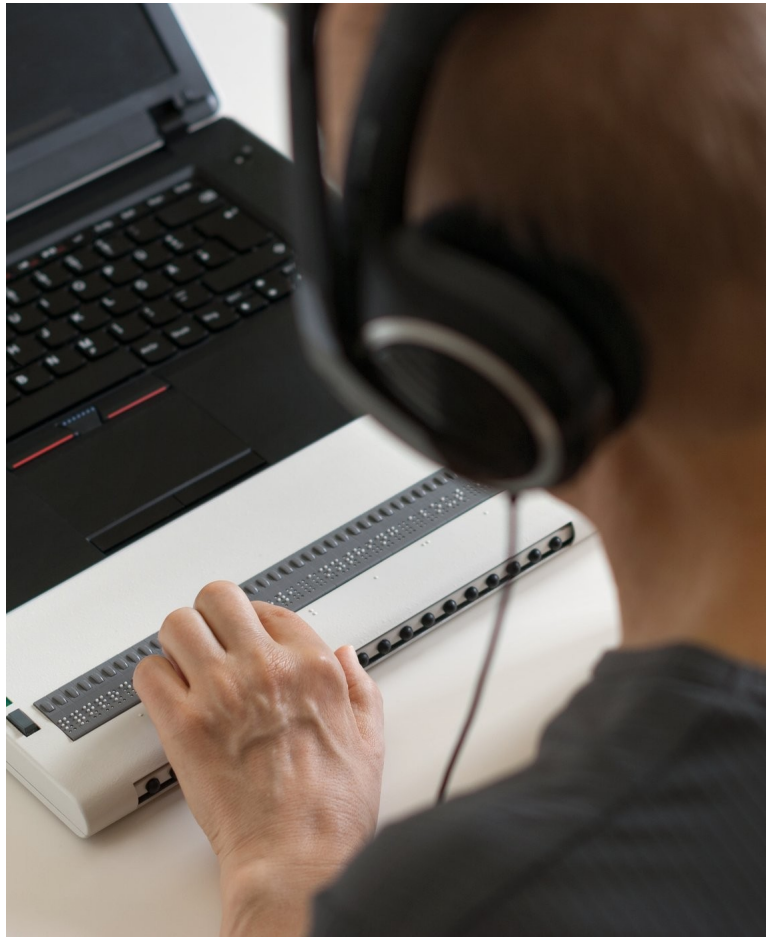
Pitfall	Why it fails	Fix
Snapshotting too early	Half-loaded page	Wait for key content before the snapshot
Dynamic content	Dates, ads, random items change every run	Use fixed test data; hide or mask dynamic areas
Different machines	Fonts render differently per OS and browser	Make baselines per browser, in the same environment that runs the tests
Whole-page snapshots	Any small change anywhere fails	Snapshot the components that matter

© 2026 by Innovation In Software Corporation

WHY ACCESSIBILITY MATTERS

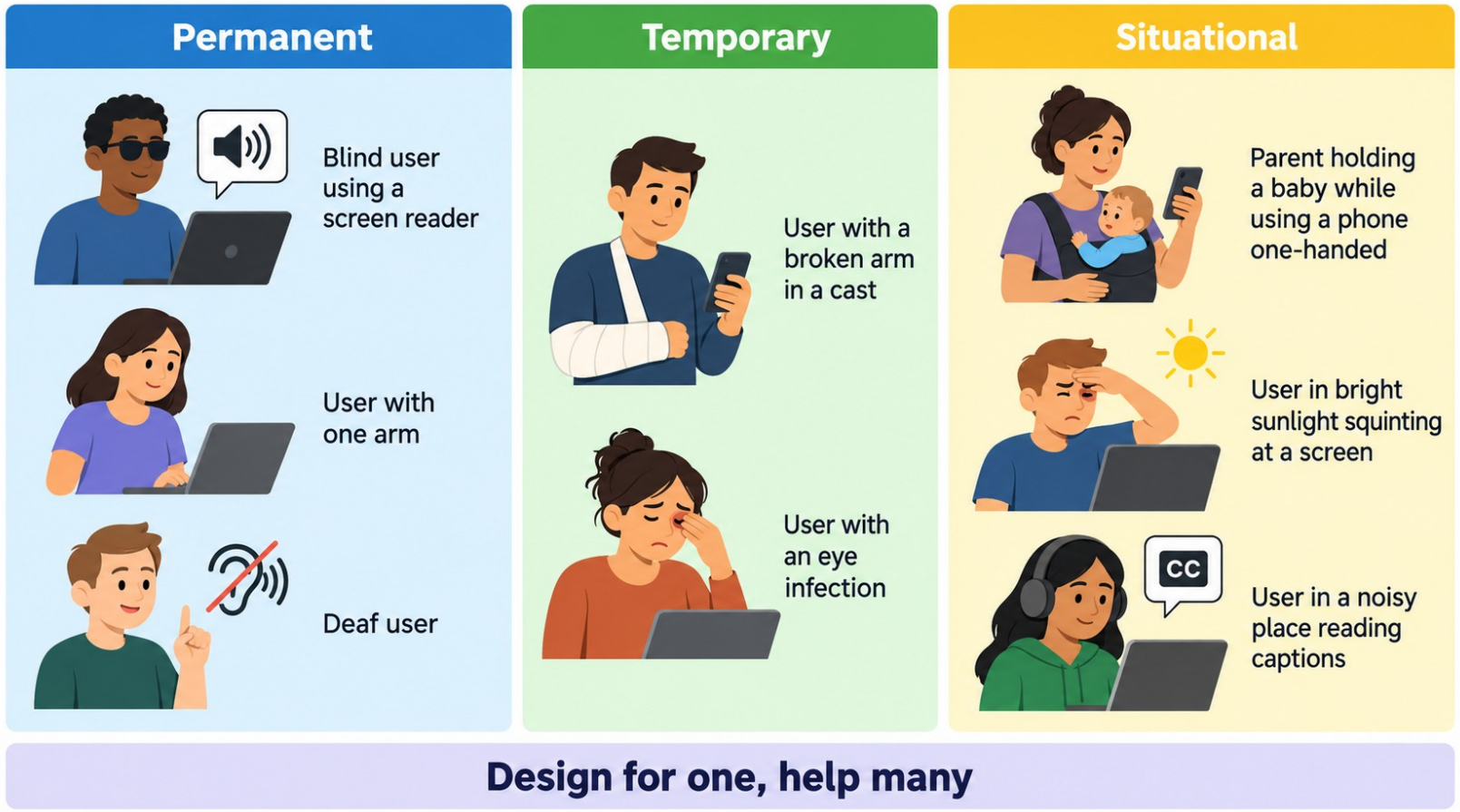
About one in six people worldwide lives with a significant disability. If your site can't be used with a keyboard, a screen reader, or low vision, those customers can't buy from you.

Accessibility is also a legal requirement in many countries and contracts. And it improves the experience for everyone: clear labels, good contrast, and keyboard support help all users.



© 2026 by Innovation In Software Corporation

WHO BENEFITS FROM ACCESSIBLE DESIGN



© 2026 by Innovation In Software Corporation

WCAG: FOUR PRINCIPLES

The **Web Content Accessibility Guidelines (WCAG)** are the global standard. Version 2.1 organizes its success criteria under four principles, often remembered as **POUR**.

Principle	Means	Example criterion
Perceivable	Users can perceive the content	1.1.1 Images have text alternatives; 1.4.3 Contrast at least 4.5:1
Operable	Users can operate the UI	2.1.1 Everything works from the keyboard; 2.4.7 Focus is visible
Understandable	Users can understand it	3.3.2 Form fields have labels or instructions
Robust	Works with assistive technology	4.1.2 Every control has a name and role

© 2026 by Innovation In Software Corporation

CONFORMANCE LEVELS

WCAG criteria come in three levels. Each level includes everything below it.

Level	Means	Typical use
A	Minimum; without it, some people can't use the site at all	Never enough on its own
AA	The standard target: removes the most common barriers	Legal and contract requirements (BookNook's NF3)
AAA	Enhanced; not achievable for all content	Specialized sites and sections

© 2026 by Innovation In Software Corporation

COMMON ACCESSIBILITY FAILURES

Most real-world failures fall into a handful of patterns. Scanners find many of them in seconds.

Failure	Who it affects	WCAG
Image with no `alt` text	Screen reader users hear a file name or nothing	1.1.1
Low color contrast	Users with low vision, anyone in bright light	1.4.3
Form field with no label	Screen reader users don't know what to type	1.3.1, 3.3.2
Controls that only work with a mouse	Keyboard and switch users	2.1.1
Buttons and links with no name	Screen readers announce just "button"	4.1.2

© 2026 by Innovation In Software Corporation

AUTOMATED SCANNING WITH AXE

axe-core is a widely used accessibility engine. The `cypress-axe` plugin runs it inside your Cypress tests.

```
cy.visit('/');
cy.injectAxe();
cy.checkA11y(null, WCAG_21_AA);
```

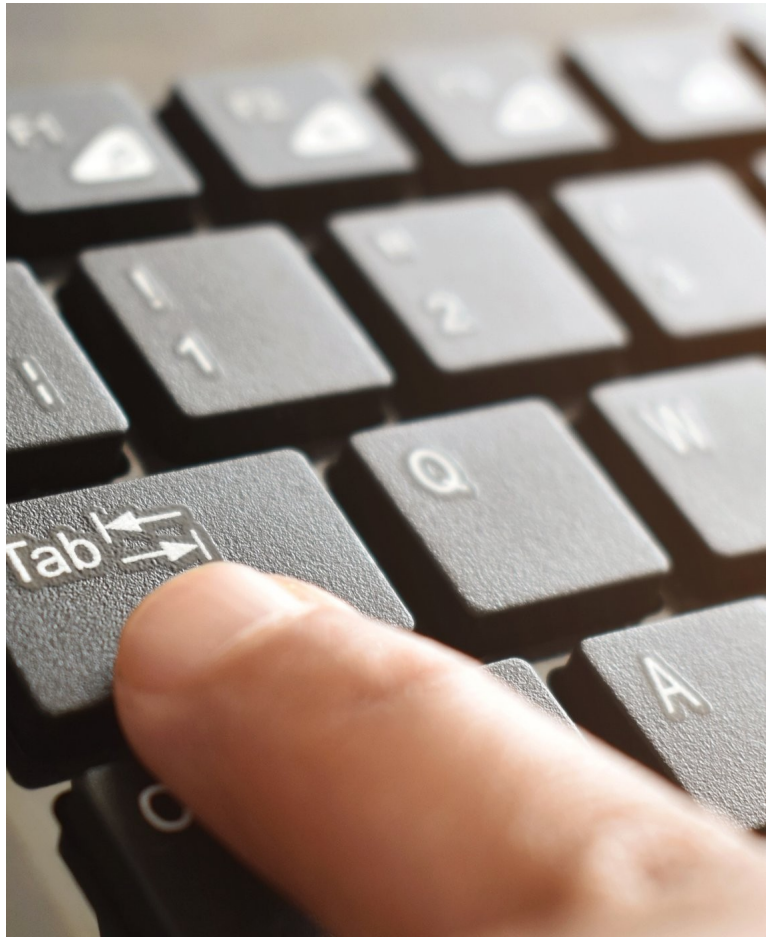
- Each violation reports a rule ID, an impact (critical, serious, moderate, minor), and the affected elements
- Map each rule back to its WCAG criterion
- Run it on every key page, in every important **state**

© 2026 by Innovation In Software Corporation

WHAT SCANNERS CAN'T CATCH

Automated scanners catch only part of the problem, roughly a third of accessibility issues. They can tell an image has no `alt` text, but not whether the `alt` text is accurate. They can't judge whether the tab order makes sense.

- **Keyboard check:** put the mouse away and complete the task with Tab, Shift+Tab, and Enter
- **Focus check:** can you always see where you are?
- **Screen reader check:** listen to the page with a screen reader
- Plan both: automated scans on every build, manual checks before release



© 2026 by Innovation In Software Corporation

STATE MATTERS FOR ACCESSIBILITY

An empty page can pass a scan that the same page fails once it has data. Inputs, buttons, and messages often appear only after the user does something.

- Scan pages in their **real** states: with items in the cart, after an error, after login
- Use API calls to set up the state quickly before the scan
- A clean scan of an empty page proves very little

© 2026 by Innovation In Software Corporation

CROSS-BROWSER TESTING

Customers don't all use your browser. Each browser renders pages and runs JavaScript with its own engine, and small differences show up as real bugs: a layout that breaks, a font that doesn't load, a feature that's missing.

Requirements should say which browsers and versions are supported. Those are the ones you test.



© 2026 by Innovation In Software Corporation

BROWSER ENGINES

Browsers that share an engine behave very similarly. Testing one per engine covers most of the risk.

Engine	Browsers	Note
Blink (Chromium)	Chrome, Edge, Opera	Most of the desktop market
Gecko	Firefox	Different rendering and some different behaviors
WebKit	Safari, and every browser on iPhone and iPad	Often the source of mobile-only bugs

Chrome and Edge share Blink, yet even they can differ slightly, for example in font smoothing.

© 2026 by Innovation In Software Corporation

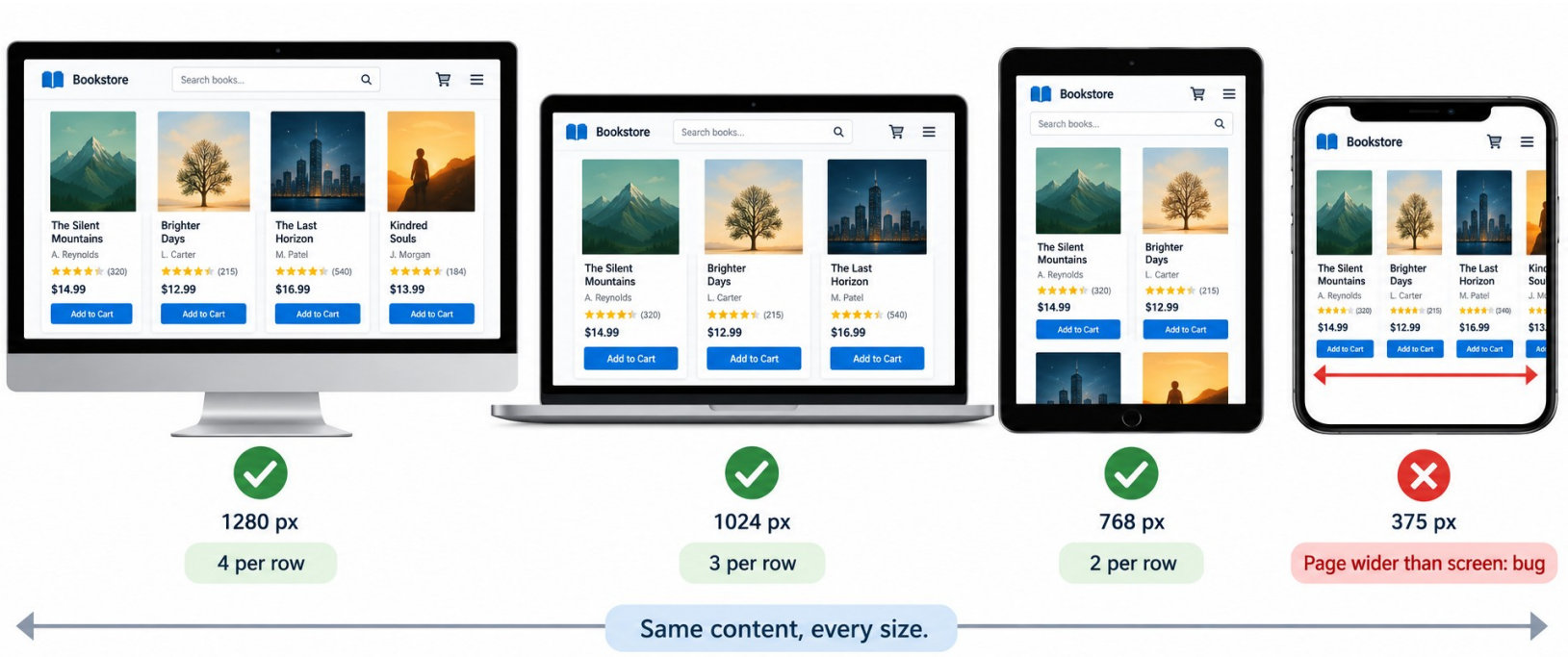
CROSS-DEVICE AND RESPONSIVE TESTING

Responsive pages rearrange themselves for each screen size. Test the sizes your users actually have, starting from analytics.

- Set the screen size in tests: `cy.viewport('iphone-x')`
- Check that key actions still work on small screens
- Check for **horizontal scrolling**, a common mobile layout bug
- Screen size isn't everything: touch, real-device performance, and mobile browsers need real devices

© 2026 by Innovation In Software Corporation

CROSS-DEVICE AND RESPONSIVE TESTING

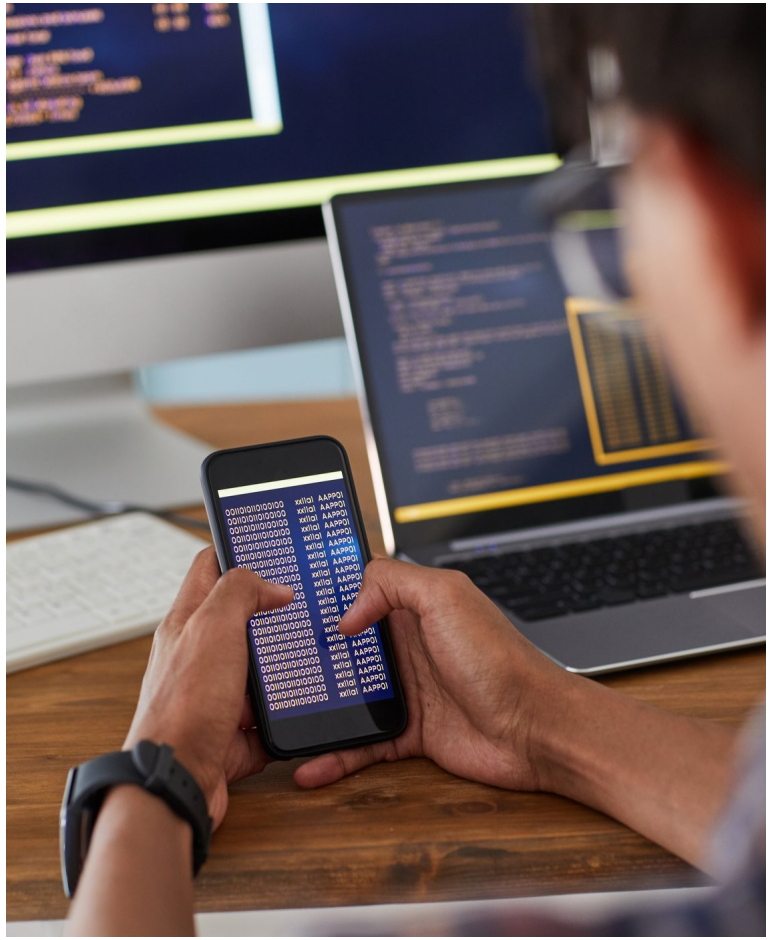


© 2026 by Innovation In Software Corporation

COVERING BROWSERS YOU CAN'T RUN

No team owns every device. When a required browser isn't available locally, there are options.

- A tool that supports it (Playwright's WebKit engine is close to Safari)
- Cloud device services that rent real browsers and phones
- A short manual pass on a real device before release
- Record the gap in the test plan; never assume it works



© 2026 by Innovation In Software Corporation

LAB 2.3 – UI TESTING: VISUAL, ACCESSIBILITY, CROSS-BROWSER

In this lab you add three new kinds of UI checks to BookNook's test suite.

- Rewrite a brittle test with robust locators, then automate search and paging
- Create a visual baseline, break it with an "innocent" CSS change, and read the diff
- Scan BookNook against WCAG 2.1 AA and do a manual keyboard check
- Run the same tests in Chrome and Edge, and on a phone-sized screen

© 2026 by Innovation In Software Corporation

POP QUIZ:

Which locator is most likely to survive a redesign of the catalog page?

A. `#results > div:nth-child(3) > button`

B. `.btn-primary`

C. `[data-cy=add-to-cart]`

D. `div div button`



POP QUIZ:

Which locator is most likely to survive a redesign of the catalog page?

A. `#results > div:nth-child(3) > button`

B. `.btn-primary`

C. `[data-cy=add-to-cart]`

D. `div div button`



POP QUIZ:

A visual test fails because the team intentionally redesigned the price label. What should you do?

A. Delete the visual test

B. Increase the threshold until it passes

C. Review the diff, then approve a new baseline

D. Report it as a bug



POP QUIZ:

A visual test fails because the team intentionally redesigned the price label. What should you do?

A. Delete the visual test

B. Increase the threshold until it passes

C. Review the diff, then approve a new baseline

D. Report it as a bug



POP QUIZ:

An automated accessibility scan passes with zero violations. What's still needed?

A. Nothing; the page meets WCAG 2.1 AA

B. Manual checks, such as using the page with only a keyboard

C. A second scan with a different browser

D. A visual regression test



POP QUIZ:

An automated accessibility scan passes with zero violations. What's still needed?

A. Nothing; the page meets WCAG 2.1 AA

B. Manual checks, such as using the page with only a keyboard

C. A second scan with a different browser

D. A visual regression test



TESTING BELOW THE UI

BookNook's six login UI tests take about 15 seconds. The team wants every cart and discount rule checked on every commit, in seconds. Most of those rules don't live in the browser at all; they live in the API behind it.

API tests skip the browser and talk to the server directly. They're faster, more stable, and they can run before the UI is even built.



© 2026 by Innovation In Software Corporation

WHAT IS AN API?

An **API** (application programming interface) is how one piece of software talks to another. When you click **Add to Cart**, the web page doesn't change the cart itself. It sends a request to BookNook's API, and the API does the work.

The same API can serve many **consumers**: the website, a mobile app, a partner's system. If the API is wrong, every one of them is wrong.



© 2026 by Innovation In Software Corporation

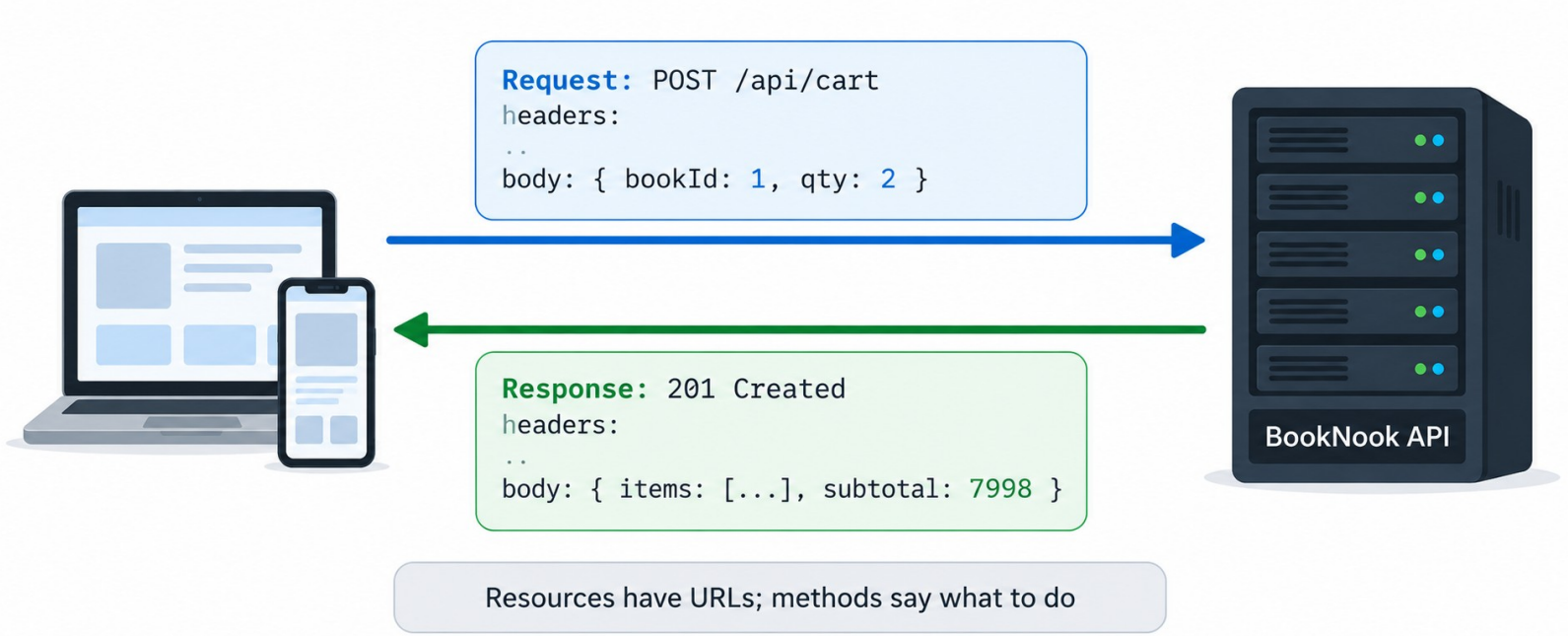
WHAT IS A REST API?

REST is the most common style for web APIs. Everything is a **resource** with a URL (a book, a cart), and you act on it with standard HTTP methods.

- The client sends a **request**; the server sends back a **response**
- Data usually travels as **JSON**

© 2026 by Innovation In Software Corporation

WHAT IS A REST API?



© 2026 by Innovation In Software Corporation

A REST CALL IN ONE TABLE

Part	What it is	BookNook example
Method	The action	`GET` read, `POST` create, `PATCH` update, `DELETE` remove
URL	The resource, plus optional query parameters	`/api/books?q=prag&page=1`
Headers	Metadata about the request or response	`Content-Type: application/json`
Body	The data, usually JSON	`{ "bookId": 1, "qty": 2 }`
Status code	The result, as a number	`201 Created`

© 2026 by Innovation In Software Corporation

HTTP METHODS

Method	Does	BookNook example	Safe to repeat?
`GET`	Reads a resource	`GET /api/books/1`	Yes, it changes nothing
`POST`	Creates something, or performs an action	`POST /api/cart` adds a book	Not always: twice may add twice
`PUT` / `PATCH`	Replaces / updates a resource	`PATCH /api/cart/1` changes quantity	Usually
`DELETE`	Removes a resource	`DELETE /api/cart/1`	Usually

"Safe to repeat" matters for testing: what happens if a user double-clicks, or a request is retried?

© 2026 by Innovation In Software Corporation

STATUS CODES

The status code is the API's one-number summary of what happened.

Code	Meaning	BookNook example
`200 OK`	Success	Search results
`201 Created`	Something new was created	Book added to the cart
`400 Bad Request`	The request is invalid	Quantity 12
`401 Unauthorized`	Not logged in, or wrong credentials	Wrong password
`404 Not Found`	No such resource	`/api/books/999`
`423 Locked`	The resource is locked	Account locked after 5 failed logins
`500 Internal Server Error`	The server crashed	Never acceptable: always a bug

© 2026 by Innovation In Software Corporation

JSON IN 60 SECONDS

JSON is the text format most APIs use. It has objects (`{ }`), arrays (`[ ]`), strings, numbers, booleans, and `null`.

```
{
  "page": 1,
  "total": 30,
  "books": [
    { "id": 1, "title": "The Pragmatic Programmer", "price": 3999, "stock": 12 }
  ]
}
```

Notice `"price": 3999`. BookNook's prices are in **cents**. Details like that belong in the API's documentation, and in your tests.

© 2026 by Innovation In Software Corporation

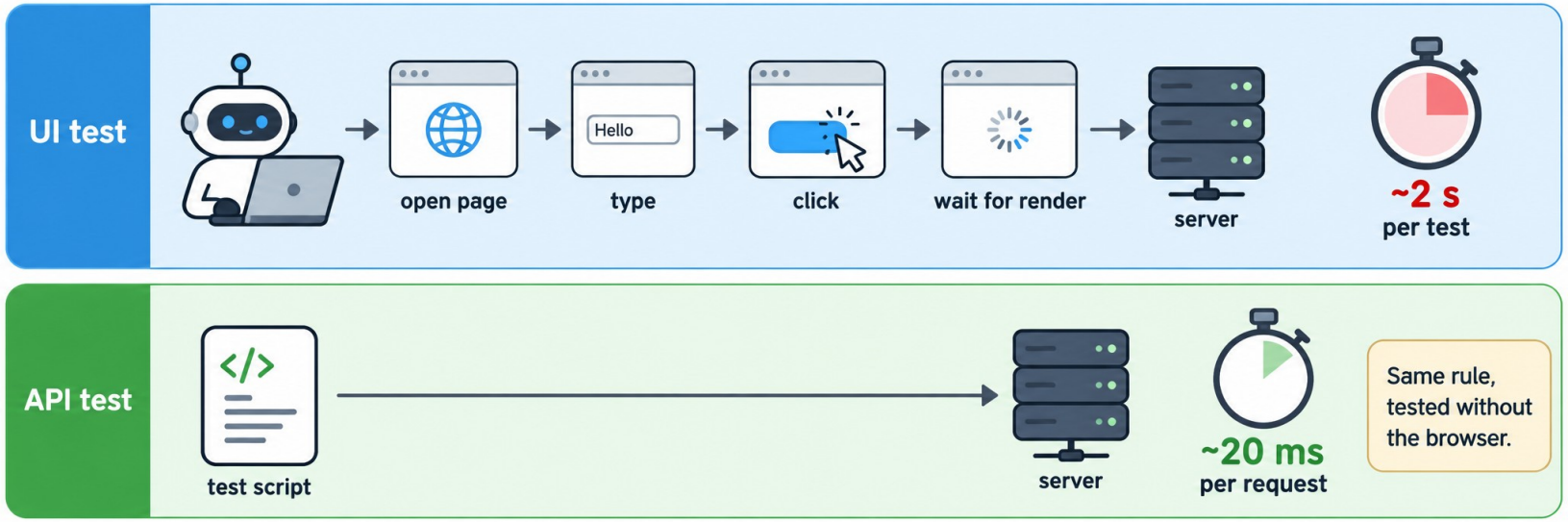
WHY TEST AT THE API LEVEL?

	UI test	API test
Speed	Seconds per test	Milliseconds per request
Stability	Breaks when the layout changes	Breaks only when the API changes
When it can start	After the UI exists	As soon as the endpoint exists, or earlier with a mock
Pinpoints	"Something in the flow broke"	"This endpoint returned the wrong value"
Can't check		What the user actually **sees**

Test business rules at the API level; keep UI tests for what only the UI can show.

© 2026 by Innovation In Software Corporation

WHY TEST AT THE API LEVEL?



© 2026 by Innovation In Software Corporation

WHAT AN API TEST CHECKS

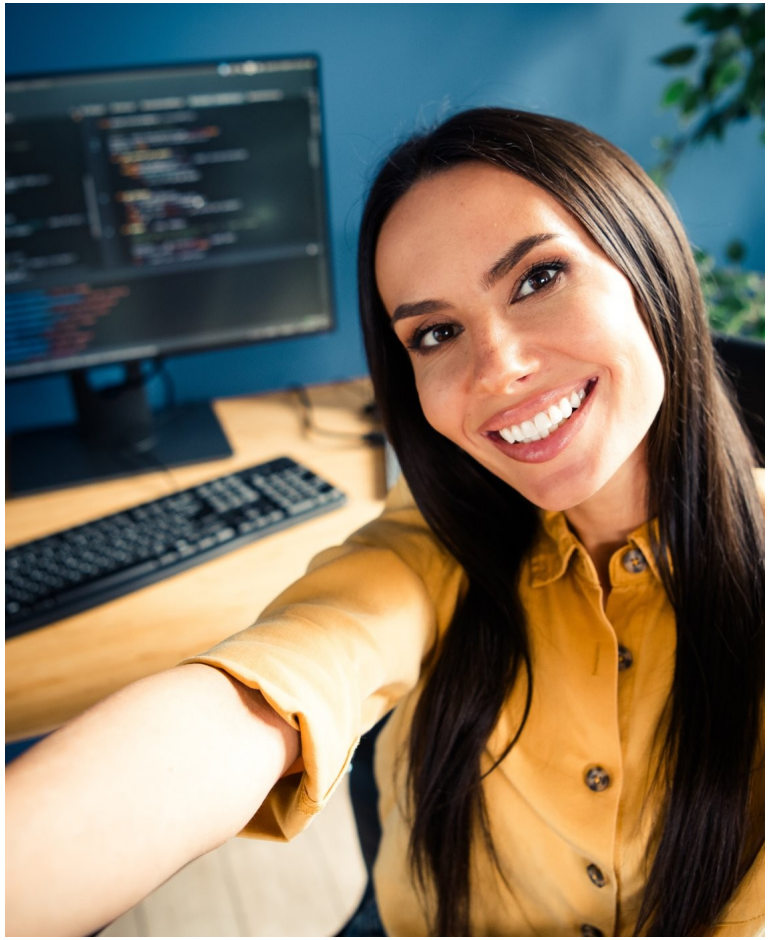
Check	Question	Example
Status	Did it succeed or fail the right way?	`201`, not `200`, after adding to the cart
Body values	Is the data correct?	`discount` is `1200` for 15% off $79.98
Headers	Is the response well formed?	`Content-Type` is JSON
Schema	Does the response have the agreed **shape**?	Every book has an integer `price`
Response time	Is it fast enough?	Under 500 ms
Side effects	Did it change what it should?	A follow-up `GET /api/cart` shows the book

© 2026 by Innovation In Software Corporation

POSITIVE AND NEGATIVE API TESTS

The API is the last line of defense. A web page can disable a button, but anyone can send any request straight to the API. Every rule must be enforced **there**.

- **Positive:** valid requests return the right status and data
- **Negative:** invalid input, missing fields, wrong types, too-large values
- **State:** out of stock, locked account, expired code
- Check that errors are **helpful**: a clear status and a clear message



© 2026 by Innovation In Software Corporation

CHOOSING AN API TOOL

	Postman	REST Assured	Code-based (Supertest, Playwright, `cy.request`)
Style	GUI, plus JavaScript test scripts	Java library, "given/when/then"	Tests in your app's language
Learning curve	Low	Medium (needs Java and a build tool)	Medium
Runs in CI	Yes, with **Newman**	Yes, with Maven or Gradle	Yes
Best fit	Exploring APIs, mixed-skill teams, sharing collections	Java teams, tests next to Java code	API and UI tests in one suite

© 2026 by Innovation In Software Corporation

POSTMAN

Postman is a desktop app for sending API requests and testing the responses. It's often the first tool a team reaches for, because anyone can read a request and its tests.

- **Requests** are grouped into **collections** and folders
- **Environments** hold values like the base URL and test accounts
- **Test scripts** in JavaScript check each response
- The **Collection Runner** runs a whole collection in order



© 2026 by Innovation In Software Corporation

REST ASSURED

REST Assured is the most popular API testing library for Java. Tests read like Day 1's acceptance criteria: given, when, then.

```
given().baseUri("http://localhost:3000").queryParam("q", "prag")
.when().get("/api/books")
.then().statusCode(200)
       .body("total", equalTo(1))
       .body("books[0].title", equalTo("The Pragmatic Programmer"));
```

It runs with JUnit, so API tests live in the same repository and build as the Java service they test.

© 2026 by Innovation In Software Corporation

WRITING TESTS IN POSTMAN

Postman tests are JavaScript that runs **after** the response arrives.

Code	Does
`pm.test(name, fn)`	Defines a named test
`pm.response.to.have.status(code)`	Checks the status code
`pm.response.json()`	Parses the body as JSON
`pm.expect(x).to.eql(y)`	Checks a value

© 2026 by Innovation In Software Corporation

VARIABLES AND ENVIRONMENTS

Hard-coding `localhost:3000` means rewriting every request to test staging. Variables fix that.

Scope	Lives in	Use for
Environment	The selected environment	Values that differ per environment: base URL, test accounts
Collection	The collection	Values shared by all requests
Data	A CSV or JSON data file	Values that change each iteration

Switch the environment, and the same collection runs against local, test, or staging.

© 2026 by Innovation In Software Corporation

CHAINING REQUESTS

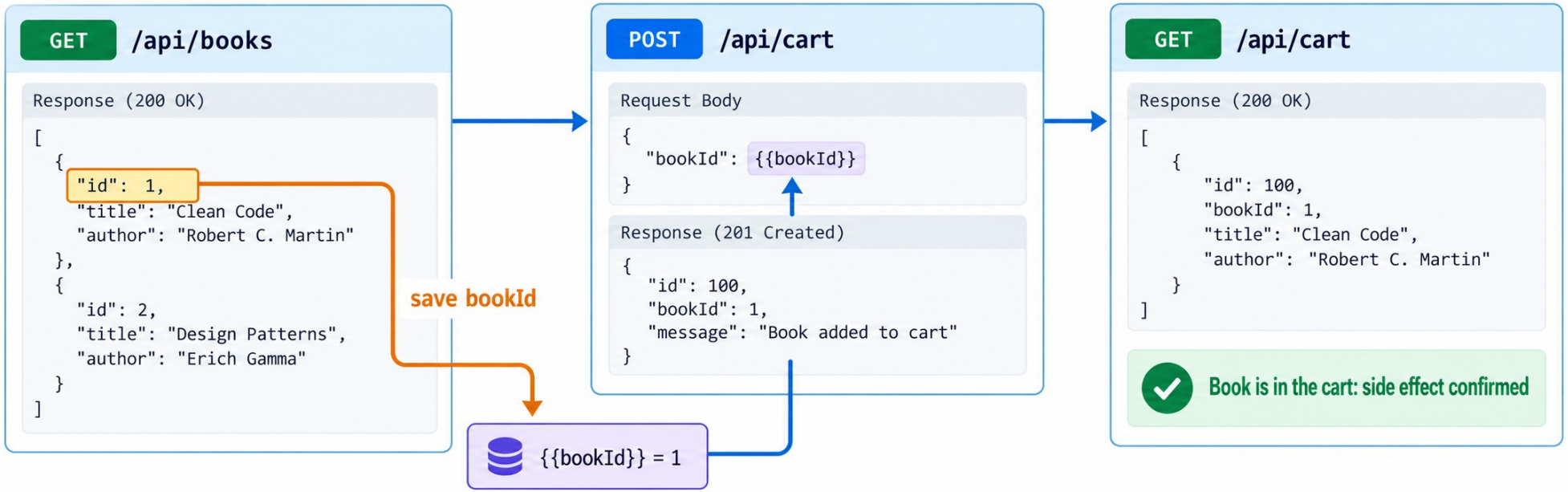
Real workflows are sequences: find a book, add it to the cart, check the cart. **Chaining** passes a value from one response into the next request.

```
const book = pm.response.json().books.find(b => b.stock > 0);
pm.environment.set('bookId', book.id);
```

The next request uses `{{bookId}}` in its body. No hard-coded IDs, so the test keeps working when the data changes.

© 2026 by Innovation In Software Corporation

CHAINING REQUESTS



© 2026 by Innovation In Software Corporation

STATE, COOKIES, AND ORDER

APIs can remember things between requests with state: a cart, a login session, a lockout count. Tests that change state must manage it.

- Start every run from a **known state**, for example with a reset request first
- Requests in a collection run **top to bottom**, so order matters
- Cookies such as a session or cart ID are sent back automatically, which is how the server recognizes you

© 2026 by Innovation In Software Corporation

DATA-DRIVEN API TESTS

Boundary testing needs many values with the same steps. Put the values in a CSV file and run the same request once per row.

qty	expectedStatus
1	201
10	201
0	400
11	400

One request, one script, as many test cases as you have rows. It's Day 1's test data table, automated.



© 2026 by Innovation In Software Corporation

RUNNING HEADLESS WITH NEWMAN

Newman runs Postman collections from the command line, with no app open. That's how API tests run in a CI pipeline.

```
npx newman run booknook.postman_collection.json -e booknook-
local.postman_environment.json
```

```
npx newman run ... --folder Quantity -d data/qty-boundaries.csv
```

```
npx newman run ... --folder Catalog -n 50
```

- `-e` chooses the environment, `-d` a data file, `-n` the number of iterations
- The summary shows requests, assertions, failures, and response times
- A non-zero exit code on failure lets a pipeline stop the build

© 2026 by Innovation In Software Corporation

A FIRST LOOK AT PERFORMANCE

A response-time assertion is a cheap early warning that runs on every commit.

```
pm.test('responds in under 500 ms', () => {
  pm.expect(pm.response.responseTime).to.be.below(500);
});
```

Running a request 50 times shows the average, minimum, and maximum time. But one user sending requests in sequence is **not** a load test. Proving "200 users at once" needs dedicated tools and a production-like environment.

© 2026 by Innovation In Software Corporation

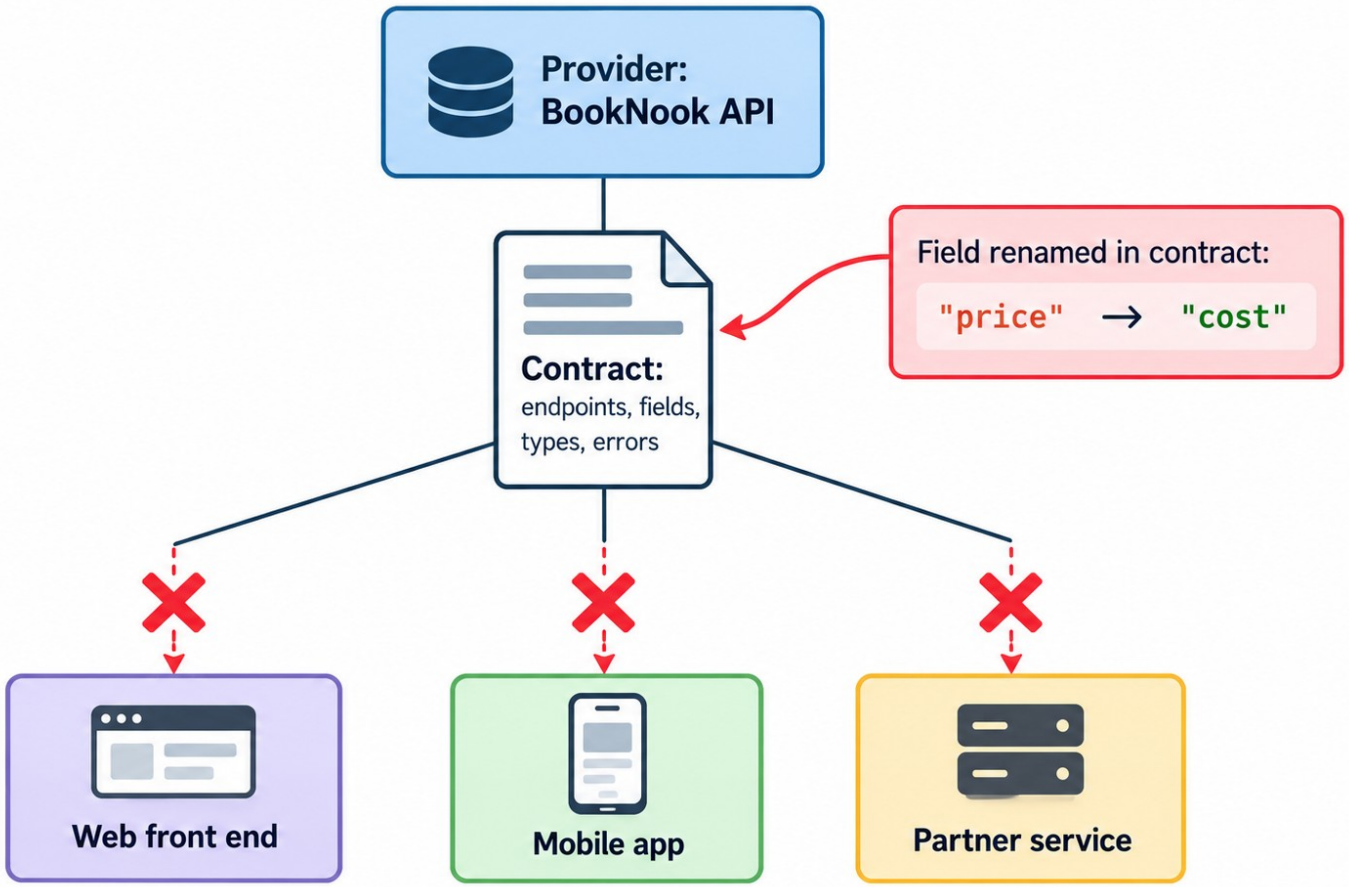
PROVIDERS, CONSUMERS, AND CONTRACTS

The team that builds an API is the **provider**. Everyone who calls it is a **consumer**: the web front end, a mobile app, another service. They depend on a **contract**: the endpoints, fields, types, and error formats both sides agree on.

When the provider changes the contract without warning, consumers break, even if the new response is "correct."

© 2026 by Innovation In Software Corporation

PROVIDERS, CONSUMERS, AND CONTRACTS



© 2026 by Innovation In Software Corporation

CONTRACT TESTING WITH JSON SCHEMA

A **JSON Schema** describes the shape of a response: which fields are required and what type each one is. Checking every response against it catches contract breaks immediately.

```
const bookSchema = {
  type: 'object',
  required: ['id', 'title', 'price', 'stock'],
  properties: {
    id: { type: 'integer' },
    price: { type: 'integer', minimum: 0 }
  }
};
pm.test('Matches the book contract', () => pm.response.to.have.jsonSchema(bookSchema));
```

© 2026 by Innovation In Software Corporation

CONSUMER-DRIVEN CONTRACT TESTING

Schema checks run on the provider's responses. **Consumer-driven contract testing** goes further: each consumer publishes exactly what it needs, and the provider's build verifies every consumer's contract before it ships.

- Tools such as **Pact** automate the exchange
- A breaking change fails the **provider's** build, not the consumer's production app
- Each team can deploy independently with confidence



© 2026 by Innovation In Software Corporation

MOCK SERVERS

A **mock server** returns saved example responses for endpoints that don't exist yet. Consumers can build and test against the agreed contract while the provider is still building the real thing.

- Agree on the contract and example responses first
- Front-end and back-end work happens in parallel
- Tests written against the mock later run against the real API



© 2026 by Innovation In Software Corporation

THE LIMITS OF MOCKS: MOCK DRIFT

A mock proves the consumer works with the **example**. It can't prove the real endpoint will match. If the provider later builds something slightly different (different field names, dollars instead of cents, a login requirement), the mock and reality have **drifted**.

The cure: run the same contract tests against both the mock and the real provider, so drift breaks a build, not production.

© 2026 by Innovation In Software Corporation

WHEN DOCS AND REALITY DISAGREE

API documentation is a requirement, and it can be wrong like any other. Comparing the docs with what the API actually does often turns up surprises: undocumented endpoints, missing fields, error formats that differ from endpoint to endpoint.

Each difference is a finding. Either the docs are wrong or the API is, and the product owner decides which.



© 2026 by Innovation In Software Corporation

LAB 2.4 – API TESTING WITH POSTMAN

In this lab you build a Postman collection that tests BookNook's API, checks its contract, and mocks an endpoint that doesn't exist yet.

- Explore the API, then write assertions on status, body, headers, and response time
- Use environment variables and chain requests from search to cart
- Write negative and data-driven tests, and match failures to Day 1 bugs
- Validate responses against a JSON Schema and compare the docs with reality
- Run the collection headless with Newman, and build a mock server for an unbuilt endpoint

© 2026 by Innovation In Software Corporation

POP QUIZ:

A request to add a book to the cart succeeds and creates a new cart item. Which status code fits best?

A. 201 Created

B. 200 OK

C. 400 Bad Request

D. 500 Internal Server Error



POP QUIZ:

A request to add a book to the cart succeeds and creates a new cart item. Which status code fits best?

A. 201 Created

B. 200 OK

C. 400 Bad Request

D. 500 Internal Server Error



POP QUIZ:

What does a JSON Schema check verify?

A. That the response is fast enough

B. That the user can see the data on screen

C. That the server is running

D. That the response has the agreed fields and types





POP QUIZ:

What does a JSON Schema check verify?

A. That the response is fast enough

B. That the user can see the data on screen

C. That the server is running

D. That the response has the agreed fields and types





POP QUIZ:

Why would a team use a mock server?

A. To make production faster

B. So consumers can build and test against an endpoint that isn't built yet

C. To replace contract testing

D. To hide bugs from the product owner



POP QUIZ:

Why would a team use a mock server?

A. To make production faster

B. So consumers can build and test against an endpoint that isn't built yet

C. To replace contract testing

D. To hide bugs from the product owner





DAY 2 WRAP-UP

Today your test cases learned to run themselves.

- TDD: write a failing test, make it pass, clean up, in small steps
- Automate what runs often and matters most, at the cheapest level that proves it
- Robust locators, visual checks, and accessibility scans cover what functional tests miss
- API tests check business rules in milliseconds; contracts and mocks keep teams in sync

Tomorrow: when tests find problems, how to triage, report, and keep the pipeline trustworthy.



© 2026 by Innovation In Software Corporation